NetLift

A Causal Decision System for Incremental Profit Optimization

Complete Project Specification

AI & Data Science — Microsoft Machine Learning Engineer Track

Digital Egypt Pioneers Initiative (DEPI) — Graduation Project

What this document is. An implementation-ready specification for an uplift modeling system: a system that predicts *who changes behaviour because of an intervention*, not who is likely to convert. It is written to survive a hostile technical question, not to sound impressive. Every dataset statistic in Part III is taken from the dataset's own published documentation and is cited at the point of use. Numbers that are arithmetic consequences of those figures are labelled as derived, not as sourced facts.

The single most important thing in this document. Accuracy, F1 and ROC–AUC are *not valid* evaluation metrics for this project, and a model that scores well on them can be worth less than random targeting. The reason is structural: the quantity this project predicts is never observed for any individual customer, so there is no per-row label to be accurate against. Section 27 explains what replaces them and Section 28 explains why the replacement metrics need confidence intervals before any result is reported. Read both before writing any training code.

Scope note. This specification is deliberately written as a *standalone* machine-learning system for a team of five. A separate frontend/backend team was considered as a combined "mega project" and that option was declined. The architecture in Part IV is nonetheless API-first (Section 35): the model is exposed as an independent service behind a documented contract, so that a web application built by anyone else could sit on top of it later without any change to the ML side. That costs the team nothing now and keeps a door open.

On curriculum coverage — read this before the proposal is submitted. This project uses no computer vision and no NLP, and Section 39 says so plainly instead of inventing a use for them. Everything else in the track is used because the problem requires it. Raise this with the instructor at the Week 7 review.

September 2026

Prepared as a technical architecture review

Contents

I Product Definition **1**

1 Project Overview **1**

1.1 Identity . . . 1

1.2 Executive summary . . . 1

1.3 The problem being solved . . . 2

1.4 Why this problem is commercially serious . . . 2

1.5 Target users . . . 3

1.6 Business value . . . 3

1.7 Why this is stronger than a churn-classification project . . . 3

1.8 Why this is stronger than a plain A/B test analysis . . . 4

2 Problem Statement **4**

2.1 Formal statement . . . 4

2.2 The fundamental problem, stated plainly . . . 4

2.3 The four customer segments . . . 4

2.4 Why this is harder than it sounds . . . 5

3 Final Recommended Product **5**

3.1 The scope decision, stated directly . . . 5

3.2 MVP — must be complete and evaluated before anything else starts . . . 6

3.3 Advanced — only after the MVP is complete and evaluated . . . 6

3.4 Future product — roadmap, not this semester . . . 6

II System Design **6**

4 Complete System Workflow **6**

5 Causal Framing and Assumptions **7**

6 Treatment Compliance: What Is Actually Being Estimated **8**

6.1 What each dataset actually supports . . . 8

6.2 The Criteo exposure column . . . 8
6.3 What that makes estimable, and at what price . . . 9
6.4 Decision . . . 10

7 Modeling Approaches 10
7.1 The candidate families . . . 10
7.2 Why the X-learner is the primary candidate here . . . 11
7.3 The class-transformation trap . . . 12
7.4 Final recommendation . . . 12

8 The Deep Learning Component 12

9 Baselines That Must Exist Before Any Uplift Model 13

10 Uplift Score to Decision: The Targeting Policy 13
10.1 The economics . . . 13
10.2 Policy modes . . . 13
10.3 Negative uplift and the sleeping-dog rule . . . 14

11 Uncertainty and Abstention 14

12 Model Output Schema 14

13 Campaign Policy Simulator 15
13.1 How to state the simulator's output without overclaiming . . . 15

14 Simulated Campaign Replay 15
14.1 Why the replay is worth building anyway . . . 15
14.2 The replay loop . . . 16
14.3 What the replay produces . . . 16
14.4 Where this shows up in the workload . . . 16

15 What a Real Deployment Would Additionally Require 16

III Data Strategy 17

16 Dataset Strategy 17
16.1 Criteo-UPLIFT — the primary dataset . . . 17
16.2 Hillstrom MineThatData — the second dataset . . . 18
16.3 Comparison . . . 18

17 The Central Data Challenge: A Thin Counterfactual **18**

18 Leakage and Validity Audit **19**

19 Splitting Strategy **20**

20 Randomization Integrity Check **20**

21 Multi-Treatment Handling **20**

22 Choosing the Outcome Variable **21**

23 Complete Data Pipeline **21**

24 Data Quality and Documentation Deliverables **22**

IV Engineering **22**

25 Training Strategy **22**

26 Hyperparameter Tuning Under an Uplift Objective **23**

27 Evaluation Metrics **23**

27.1 Why the usual metrics do not apply 23

27.2 The metric set 23

28 Statistical Significance — The Section That Protects Every Claim **24**

29 Error Analysis and Segment Audit **24**

30 Behavioural and Invariance Tests **24**

31 Explainability **25**

32 Concept Drift and Monitoring **25**

32.1 Delayed feedback — a deployment problem, not one this project faces 25

33 Experiment Tracking and Reproducibility **26**

34 Testing **26**

35 Deployment Architecture **26**

35.1 API design 26

35.2 Serving stack 27

36 Technology Stack **28**

37 Repository Structure **28**

38 Hardware **29**

39 Curriculum Coverage — Stated Honestly **29**

V Product and Demonstration **30**

40 User Interface **30**

40.1 Tab 1 — Campaign Planner 30
40.2 Tab 2 — Policy Comparison 30
40.3 Tab 3 — Model Evidence 30
40.4 Tab 4 — Monitoring 30
40.5 Customer detail view 30

41 End-to-End Demo Scenario **30**

42 What the Demo Must Prove **31**

VI Execution **31**

43 Team Distribution **31**

44 Integration Plan **33**

45 Meeting the DEPI Documentation Requirements **33**

46 Key Performance Indicators **34**

47 Milestones **36**

47.1 Milestone 1 — Team Formation and Planning (Week 5) 36
47.2 Milestone 2 — Instructor Review (Week 7) 36
47.3 Milestone 3 — Requirements Gathering (Weeks 9–10) 37
47.4 Milestone 4 — Analysis and Design (from Week 12) 37
47.5 Milestone 5 — Implementation 38
47.6 Milestone 6 — Testing and Quality Assurance 38

47.7 Milestone 7 — Release and Final Presentation (Week 21) 39

48 Project Timeline **39**

49 Scope Control **40**

49.1 MUST HAVE 40

49.2 SHOULD HAVE 40

49.3 NICE TO HAVE 41

49.4 REMOVE IF BEHIND SCHEDULE, in this order 41

50 Business Model **41**

51 Competitive Advantage **42**

52 Limitations **42**

53 Ethical and Privacy Considerations **43**

54 Risks and Difficulties **43**

54.1 Top three 44

55 Anticipated Defence Questions **44**

56 Final Deliverables **45**

57 Final Architecture Diagram **45**

58 Final Evaluation **46**

58.1 Scores 46

58.2 Top strengths 47

58.3 Top weaknesses 47

58.4 Do I recommend building it? 47

58.5 Is the scope realistic for five students? 47

58.6 What to remove if the project becomes too large 48

59 Final Recommended Version **48**

Part I

Product Definition

1 Project Overview

1.1 Identity

Name	NetLift
One line	A targeting engine that ranks customers by the *incremental* effect an offer will have on them, and refuses to spend budget on customers who would have converted anyway — or who react badly to being contacted.
Positioning	*A decision system for incremental profit optimization built on causal machine learning* — not "an uplift model". The distinction is not marketing. A model produces a score; this system closes a loop: it scores, allocates a budget, assigns treatment while holding back a control group, and then measures what that decision actually produced — replayed over a held-out slice of historical experimental data, because that is what the available datasets allow (Section 14). Describe it this way in the report, in the presentation and on a CV.
Domain	Business analytics / marketing science, directly transferable to banking, telecom, e-commerce and FinTech retention teams.
Core ML problem	Estimation of the Conditional Average Treatment Effect (CATE) from randomized-experiment data; a causal-inference problem, not a classification problem.
Primary data	Criteo-UPLIFT (large-scale randomized advertising experiment) and Hillstrom MineThatData (small, clean, three-arm email experiment).

1.2 Executive summary

Every company with a retention or promotion budget faces the same question and almost all of them answer it wrongly. They build a model that predicts *who will churn* or *who will buy*, sort customers by that score, and send the offer to the top of the list. The model is often accurate. The campaign still loses money.

It loses money because the score being optimised is the wrong quantity. A customer at the top of a "likely to buy" list may be a customer who was going to buy regardless — the discount is pure margin given away. A customer at the top of a "likely to churn" list may be someone whose decision to leave is already final — the retention call is wasted. And a specific, well-documented minority of customers respond to being contacted by leaving: a dormant subscriber reminded that they are still paying for something they forgot about.

Uplift modeling asks the question the business actually needs answered: *for this customer, how*

much does the outcome change because we intervened? Formally, it estimates

$$\tau(x) \;=\; \mathbb{E}[Y(1) \mid X = x] \;-\; \mathbb{E}[Y(0) \mid X = x]\,,$$

the difference between what happens if the customer is treated and what happens if they are not.

NetLift is a complete system built around that estimate: a validated data pipeline over randomized-experiment data, a set of uplift models benchmarked against the response model a business would otherwise use, an evaluation harness built on Qini and AUUC rather than accuracy, a decision layer that converts uplift scores into a budget-constrained targeting policy with an explicit abstention path, and a deployed service and dashboard that report the outcome in the only unit that matters to the customer of this system: incremental conversions and incremental profit per unit of campaign spend.

1.3 The problem being solved

A retention or acquisition campaign has a fixed budget. Contacting a customer has a cost: a discount that reduces margin, an SMS or call-centre minute, or the finite attention of a customer who can be contacted only so often before they disengage. The business must choose a subset of customers to contact.

The industry-standard method for making that choice is a *response model* or a *churn model*. Both are classifiers, both are trained on outcomes among contacted customers, and both rank by predicted outcome probability. Neither of them contains any information about whether the contact caused the outcome. They are answering "who is likely to convert?" when the business needs "whose conversion depends on us?"

1.4 Why this problem is commercially serious

The gap between the two questions is not academic. A campaign targeted by response model can show a strong measured conversion rate among the contacted group and still produce close to zero incremental revenue, because the contacted group was selected precisely for containing the customers most likely to convert unprompted. The company sees a good-looking campaign report and a flat revenue line, and cannot explain the contradiction — because the report never measured the counterfactual.

An uplift-targeted campaign is measured differently and can be defended differently. The claim it makes is of the form: *at the same budget, this policy produced N more conversions than targeting the same number of customers at random, and M more than targeting by response score.* That is a statement a finance function can audit.

1.5 Target users

User	What they do with it
Retention / CRM analyst	Uploads or selects a customer base, sets a budget and a per-contact cost, receives a ranked target list and an expected incremental-conversion figure with an uncertainty range.
Marketing manager	Compares candidate policies (uplift vs. response model vs. random vs. blanket contact) on one screen before committing spend.
Data science team	Consumes the scoring API directly, inspects the Qini curves and drift monitors, retrains on a schedule.
Finance / campaign audit	Reads the incremental-profit report and the size of the randomized holdout that produced it.

1.6 Business value

- **Budget reduction at constant outcome.** If the persuadable population is a minority of the base — which is the normal case — most of the contact budget is currently spent on customers whose behaviour does not change. Removing them reduces cost without reducing incremental conversions.
- **Avoided negative effects.** Suppressing customers with negative estimated uplift protects revenue that a conventional campaign actively destroys.
- **Auditable causal claims.** The system's headline number is produced against a randomized control group, which means it is a measurement, not a projection.
- **Reusability.** The same machinery applies to retention offers, cross-sell, credit-limit increases, collections contact strategy and fee waivers. It is a capability, not a campaign.

1.7 Why this is stronger than a churn-classification project

	Standard churn project	NetLift
Question answered	Who is likely to leave?	Whose decision changes if we act?
Label	Observed for every row	*Never* observed for any row (Section 2.2)
Data requirement	Any historical data	Randomized experiment, or explicit and defended assumptions in its absence
Evaluation	Accuracy, F1, ROC–AUC	Qini coefficient, AUUC, uplift@k, all with confidence intervals
Failure mode	Model is inaccurate	Model is accurate *and* the campaign still loses money
What the defence panel can ask	"What was your F1?"	"How do you know the effect you measured is not noise?" — a question with a real answer here

The practical consequence: a churn classifier is a well-understood exercise that hundreds of graduation projects complete every year. The moment the target becomes a treatment effect, four things change at once — the label becomes unobservable, the metric set changes, the data

requirement becomes an experiment, and the decision layer acquires a cost structure. Those four changes are the substance of this project.

1.8 Why this is stronger than a plain A/B test analysis

An A/B test answers "did the offer work *on average*?". That is a single number for the whole population and it is frequently near zero even when the offer works very well on a subgroup and backfires on another. Uplift modeling is the conditional version of the same question, and the conditional version is the one that can be acted on: it produces a *policy*, not a verdict.

2 Problem Statement

2.1 Formal statement

For each customer i with covariates X_i, let $T_i \in \{0, 1\}$ denote treatment (contacted or not) and let $Y_i(1)$ and $Y_i(0)$ be the potential outcomes under treatment and under control. The individual treatment effect is $\tau_i = Y_i(1) - Y_i(0)$. The estimand is the conditional average treatment effect

$$\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \,.$$

The observed outcome is $Y_i = T_i\, Y_i(1) + (1 - T_i)\, Y_i(0)$.

2.2 The fundamental problem, stated plainly

> For any given customer, exactly one of $Y_i(1)$ and $Y_i(0)$ is ever observed. The quantity the model is asked to predict, τ_i, is therefore **never available as a training label for any single row in any dataset that has ever existed**. This is not a data-collection shortcoming that a better dataset would fix; it is the structure of the problem. Every consequence in this document — the choice of estimators, the metric set, the need for confidence intervals, the holdout a deployment would need — follows from this one sentence.

What *is* recoverable is the effect for a *group*. If treatment was assigned at random, then within any group defined by pre-treatment covariates, the treated and untreated members are exchangeable, and the difference in their average outcomes is an unbiased estimate of the group's average treatment effect. Uplift modeling is the discipline of forming those groups well.

2.3 The four customer segments

The population partitions conceptually into four types, and two of them cost money when targeted. The names below are the standard ones in the causal-targeting literature.

Segment	If treated	If not treated	Targeting implication
Persuadables	Converts	Does not convert	**The only group worth spending on.** Positive uplift.
Sure Things	Converts	Converts	Zero uplift. Every unit of budget here is a discount given to someone who needed no discount.
Lost Causes	Does not convert	Does not convert	Zero uplift. Wasted contact.
Sleeping Dogs	Does not convert	Converts	**Negative uplift.** Contacting them destroys value that would otherwise have existed.

Source: the four-segment framing (persuadables, sure things, lost causes, sleeping dogs) is the standard formulation used in the unit-selection literature; see Li & Pearl, Unit Selection Based on Counterfactual Logic, *IJCAI-19, as referenced in the CausalML documentation.*

A response model cannot distinguish Persuadables from Sure Things, because both convert when treated and the response model only ever sees treated conversions. This single confusion is the reason response-model-driven campaigns overspend, and it is the clearest one-sentence justification for the project.

2.4 Why this is harder than it sounds

1. **The signal is small relative to the outcome.** Conversion probability might be driven by strong, obvious features. The *change* in conversion probability caused by an email is typically a fraction of a percentage point. Any method that estimates uplift by subtracting two separately-fitted outcome models is subtracting two large, noisy numbers to recover a small one.
2. **There is no per-row validation.** A model cannot be checked one customer at a time. Validation is necessarily population-level and therefore statistical, which is why Section 28 exists.
3. **Treatment groups are usually unbalanced.** In the primary dataset of this project the treated share is 85%, leaving a comparatively thin control group that carries the entire counterfactual estimate.
4. **The metric can be overfitted.** Selecting hyperparameters by maximising a Qini score on the same data used to report it produces exactly the kind of inflated result this project exists to argue against.

3 Final Recommended Product

3.1 The scope decision, stated directly

NetLift's MVP is **single-treatment, binary-outcome uplift modeling on randomized experimental data, with a budget-constrained decision layer and a deployed scoring service**. Multi-treatment optimisation, revenue (continuous-outcome) uplift, and deep causal architectures are Advanced tier. Observational (non-randomized) causal inference is explicitly out of scope for the semester, for the reason given in Section 5.

3.2 MVP — must be complete and evaluated before anything else starts

- Validated ingestion of Criteo-UPLIFT and Hillstrom, including a randomization-integrity check (Section 20).
- Three mandatory baselines: random targeting, blanket targeting, and a response model.
- T-learner and X-learner uplift estimators over gradient-boosted trees.
- Evaluation harness: Qini curve, Qini coefficient, AUUC, uplift@k, cumulative-gain curves, each with bootstrap confidence intervals.
- Decision layer: budget-constrained top-k targeting with negative-uplift suppression and an explicit "insufficient evidence" outcome.
- MLflow experiment tracking; reproducible runs from a config file.
- FastAPI scoring service, Dockerised; Streamlit dashboard; test suite.

3.3 Advanced — only after the MVP is complete and evaluated

- Uplift Random Forest with KL / Euclidean / χ^2 split criteria as a model-class challenger.
- A shared-representation neural estimator (TARNet / Dragonnet family) as the deep-learning component (Section 8).
- Multi-treatment uplift on Hillstrom's two email arms.
- Revenue uplift on Hillstrom's continuous `spend` outcome.
- Azure ML managed endpoint deployment and drift monitoring.

3.4 Future product — roadmap, not this semester

- Observational uplift with propensity modelling and sensitivity analysis for unmeasured confounding.
- Continuous / dose-response treatment (how large a discount, not whether to discount).
- Online policy learning with a permanently maintained randomized holdout.
- Budget allocation across multiple simultaneous campaigns.

Part II

System Design

4 Complete System Workflow

```
RANDOMIZED EXPERIMENT DATA  (features X, treatment T, outcome Y)
            |
            v
[1] INGESTION + SCHEMA CONTRACT
```

```
          |
          v
[2] RANDOMIZATION INTEGRITY CHECK      -- fails? stop. do not model. (S 20)
    standardized mean differences, treated vs control, per covariate
          |
          v
[3] LEAKAGE AUDIT                      -- post-treatment features removed (S 18)
          |
          v
[4] STRATIFIED SPLIT on (treatment x outcome) jointly     (S 19)
    train / validation / LOCKED test
          |
          v
[5] BASELINES FIRST                    -- random, blanket, response model (S 9)
          |
          v
[6] UPLIFT ESTIMATORS                  -- T-learner, X-learner, uplift forest (S 7)
          |
          v
[7] EVALUATION HARNESS                 -- Qini / AUUC / uplift@k + bootstrap CI
          |                               (S 27, S 28)
          v
[8] DECISION LAYER                     -- budget, cost, margin -> policy   (S 10)
    negative-uplift suppression | abstention when CI straddles zero
          |
          v
[9] SERVING                            -- FastAPI: batch scoring + single scoring
          |                               MLflow registry, Docker, Azure
          v
[10] DASHBOARD                         -- Qini curves, segment profile, ROI simulator
          |
          v
[11] SIMULATED CAMPAIGN REPLAY         -- decide on a held-out slice without
     over the held-out test split          reading outcomes, then reveal and
                                           measure what the decision produced (S 14)
```

The order of blocks 2–5 is not stylistic. Each is a gate: a failure at block 2 invalidates everything downstream, and a model trained before block 5 exists has nothing to be compared against and therefore no defensible claim attached to it.

5 Causal Framing and Assumptions

Uplift estimates are only meaningful under stated assumptions. Naming them is not academic decoration — it is the difference between a project that can defend its numbers and one that cannot.

Assumption	Status in this project
Unconfoundedness $\{Y(1), Y(0)\} \perp T \mid X$	**Holds by construction.** Both datasets come from randomized experiments, so treatment assignment is independent of potential outcomes by design. This is the entire reason for choosing experimental rather than observational data, and it is what makes the semester feasible.
Overlap / positivity $0 < P(T{=}1 \mid X) < 1$	Holds under randomization, but must still be *verified empirically* in the primary dataset, because a treated share of 85% means some covariate regions may contain very few control rows.
SUTVA (no interference; one version of treatment)	Assumed. Worth stating aloud that in a real deployment it can fail — a customer who receives an offer may tell a friend who did not. Not testable here; declared as a limitation.
No post-treatment features	Enforced by the audit in Section 18. This is the assumption most likely to be violated by accident.

Why observational uplift is out of scope. Dropping the randomization assumption means the team must additionally estimate propensity scores, defend the absence of unmeasured confounders, and run sensitivity analyses — and, critically, must do all of that with no way of checking whether it worked, because there is no ground-truth effect to compare against. That is a research programme, not a semester deliverable. Choosing randomized data is not the easy option being taken to avoid difficulty; it is the option that makes the results *verifiable*. Say it in exactly those terms in the defence.

6 Treatment Compliance: What Is Actually Being Estimated

Correction, 1 October 2026. An earlier version of this section stated that neither dataset provides an exposure indicator, and built the case for intention-to-treat estimation on that claim. **The claim was false.** Criteo-UPLIFT carries an `exposure` column. It was found independently in two tasks during dataset verification, and confirmed against the file header. The section below is rewritten around what the data actually contains. The conclusion — that the MVP reports ITT — survives, but it now rests on a reason that is true.

Assigned treatment and received treatment are not the same thing. An email is sent; it lands in a spam folder, or is never opened. An advertisement is served; the user never looks at it. A customer assigned to control encounters the same offer through another channel. In every one of these cases the customer is recorded as treated while receiving nothing, or as control while effectively being treated.

6.1 What each dataset actually supports

Estimand	What it means here
Intention-to-Treat (ITT)	The effect of *being assigned* to the campaign. Both datasets support it, and it is the quantity a marketing team actually controls — they decide who to send to, not who opens. This is what the MVP reports.
Treatment-on-the-Treated	The effect on those who genuinely received the intervention. Larger than ITT whenever compliance is imperfect. **Not identifiable in Hillstrom**, which carries no exposure indicator. **Identifiable in Criteo** under the assumptions set out below, because Criteo carries an `exposure` column.
Contamination	Control-group members reached through another channel. Biases the estimate *downwards*, making the intervention look weaker than it is. Not detectable in either dataset.

6.2 The Criteo exposure column

Counted from the file by `scripts/criteo_complier_baseline.py`, which prints the raw 2×2 table so that every figure in this section can be traced to a count rather than to a published ratio. The row totals agree with `docs/unit-of-observation.md`.

	`exposure` = 0	`exposure` = 1	**Total**
`treatment` = 0	2,096,937	0	2,096,937
`treatment` = 1	11,454,443	428,212	11,882,655
Total	13,551,380	428,212	13,979,592

Compliance, $P(\text{exposed} \mid \text{treated})$, is therefore 3.6037%, with a 95% bootstrap interval of [3.5931%, 3.6142%].

Source: counts produced by `scripts/criteo_complier_baseline.py` (seed 42); the `exposure` column is documented on the Criteo AI Lab dataset page and reached through `scikit-uplift`'s `fetch_criteo`. Cross-checked against `docs/unit-of-observation.md` and `reports/t16_dataset_verification.md`.

An earlier draft of this table gave 11,882,653 treated rows. That figure is round(13,979,592 × 0.85) — the published treatment ratio applied to the published row count, not a count. The counted value is 11,882,655. No conclusion depends on the difference, but the distinction between a measured and a derived figure is one this document

demands of everyone, so it is recorded rather than quietly repaired.

Two properties follow, and both matter.

`exposure` is post-treatment. It is a consequence of assignment, not a characteristic of the customer. It belongs on the forbidden-column list and must never enter the feature matrix (Section 18). A model given this column would learn that exposed users convert more often, which is an association produced by the treatment itself, not a basis for deciding whom to target.

Non-compliance is one-sided. No control row is exposed, so $P(\text{exposed} \mid \text{control}) = 0$. In instrumental-variable terms there are no defiers and the monotonicity condition holds *by construction* — a condition that normally needs defending, and here does not.

This matters for knowing which assumption to argue about. Monotonicity is not the load-bearing one. The exclusion restriction is, and it is stated and examined below.

6.3 What that makes estimable, and at what price

With assignment as the instrument and exposure as the treatment actually received, the treatment-on-the-treated effect among compliers is the Wald ratio:

$$\widehat{\text{ToT}} = \frac{\widehat{\text{ITT}}}{P(\text{exposed} \mid \text{treated})} = \frac{\widehat{\text{ITT}}}{0.036037} \approx 27.7 \times \widehat{\text{ITT}}$$

This identifies the effect on compliers only — users who were exposed because they were assigned — and it requires the **exclusion restriction**: that assignment affects conversion *only* through exposure. Being eligible for targeting but never shown the advertisement must have no effect of its own.

What the Wald ratio gives, measured. Produced by `scripts/criteo_complier_baseline.py` (seed 42, 20,000 bootstrap draws). Intervals are 95% bootstrap percentile intervals.

Quantity	Estimate	95% interval
Control conversion rate	0.1938%	[0.1878%, 0.1998%]
Treated, exposed	5.3784%	[5.3108%, 5.4467%]
Treated, unexposed	0.1194%	[0.1174%, 0.1214%]
ITT	0.1152 pp	[0.1084, 0.1219]
ToT (Wald), among compliers	3.1964 pp	[3.0095, 3.3824]
$E[Y(0) \mid \text{complier}]$	2.1820%	[2.0100%, 2.3575%]
Relative lift among compliers	+146% (2.46× the baseline)	[+128%, +168%]

The denominator is the point. A complier's untreated baseline is not the population control rate. Under one-sided non-compliance, `exposure` = 1 inside the treated arm identifies compliers and `exposure` = 0 identifies never-takers, so the exclusion restriction — already required by the Wald ratio — identifies the compliers' baseline by subtraction:

$$E[Y(0) \mid \text{complier}] = \frac{\bar{y}_{\text{control}} - (1-\pi)\,\bar{y}_{\text{treated, unexposed}}}{\pi}, \qquad \pi = P(\text{exposed} \mid \text{treated}).$$

Against that denominator the effect is a 146% relative increase on a 2.18% base: exposed compliers convert at 5.38%, 2.46 times their untreated baseline. None of the 20,000 bootstrap draws placed the baseline below zero.

Correction, 10 October 2026. An earlier version of this subsection divided the Wald estimate by the *population* control rate, obtained a relative lift of about sixteen-fold, called it implausible, and concluded — in this box, in the decision table, and in the limitations list (Section 52) — that the exclusion restriction fails here. All three said so before the analysis that would establish it existed.
The denominator was wrong. A Wald estimate is an effect among compliers; dividing it by a mean taken over compliers and never-takers together compares two quantities that describe different people. Measured, compliers carry a baseline of 2.1820% against 0.1194% for never-takers, a factor of about eighteen. Dividing by the population mean of 0.1938% understated the denominator by a factor of about eleven (2.18% against 0.19%), and the sixteen-fold figure was that arithmetic, not a property of the data.

Caught in review by Ali Alaa, who supplied the estimator above.

What the measurement settles, and what it does not. It removes the evidence previously claimed *against* the exclusion restriction. It does not establish that restriction, which is not testable here.
The one-sided premise is not an assumption here: it rests on the count of zero exposed control rows in the table above. The exclusion restriction — that assignment reaches conversion by no route other than exposure — is the assumption the estimate rests on, and nothing in this data can check it.

All intervals in this section are bootstrap percentile intervals that treat the 13,979,592 rows as independent. Criteo carries no user identifier, so independence cannot be verified (Section 24 and `docs/unit-of-observation.md`); if rows are clustered by user, these intervals are too narrow. The point estimates are not affected.

6.4 Decision

Tier	What is done
MVP	ITT only, on both datasets. The targeting decision is an assignment decision, so ITT is the quantity the product actually optimises.
Advanced	The Wald estimate on Criteo, reported beside ITT, with the compliance rate, the exclusion restriction stated explicitly, and the relative lift expressed against the compliers' own untreated baseline rather than the population control rate. Reporting it against the population mean is the error this section was corrected for.
Never	Three things, and the second is the one a team actually does by accident. **(a)** `exposure` as a model feature. **(b)** `exposure` substituted for `treatment` as the treatment variable in an uplift model — this looks like an obvious upgrade and is the single worst move available, because exposure among the treated is self-selected, so it throws away the randomization the whole project rests on. **(c)** A direct comparison of exposed users against control, which is confounded by exactly the selection the experiment was designed to remove.

The sentence to use: "We report the intention-to-treat effect — the effect of being selected for the campaign — because that is the decision the product actually makes. Criteo also carries an exposure indicator and compliance is 3.60%, so we additionally report the treatment-on-the-treated effect as a Wald ratio: 3.20 percentage points among compliers, a 146% relative increase over the compliers' own untreated baseline of 2.18%. We quote it against that baseline and not against the population control rate, because compliers are a self-selected 3.6% whose baseline is about eighteen times the never-takers'; dividing by the population mean inflates the effect-to-baseline ratio about eleven-fold. The estimate rests on the exclusion restriction, which we cannot test." Saying this before being asked is worth more than any modelling choice in the document, because it demonstrates that the team knows what its own numbers mean.

7 Modeling Approaches

7.1 The candidate families

Approach	How it works	Principal weakness	Verdict here
S-learner (single model)	One model $\mu(x,t)$ trained on all data with treatment as an input feature; uplift is $\mu(x,1) - \mu(x,0)$.	When the treatment effect is small, regularisation can shrink the treatment feature's influence toward zero and the model predicts near-constant uplift. With a 0.29% conversion rate this is a realistic failure, not a theoretical one.	Include as a *diagnostic* baseline; expect it to underperform and report why.

Approach	How it works	Principal weakness	Verdict here
T-learner (two models)	Separate models $\mu_1(x)$ on treated and $\mu_0(x)$ on control; uplift is $\mu_1(x) - \mu_0(x)$.	Subtracts two independently-fitted, individually-noisy estimates; the variance of the difference can exceed the effect being estimated. The control model is fitted on the smaller group and is therefore the weaker of the two.	**Primary MVP baseline.** Simple, transparent, and the honest thing to beat.
X-learner	Uses each group's model to impute the missing counterfactual for the other group, fits effect models on the imputed effects, and combines them weighted by the propensity score.	More moving parts; more places to leak if cross-fitting is implemented carelessly.	**Primary candidate.** Specifically designed for the unbalanced-group case, which is exactly this dataset (Section 7.2).
R-learner	Orthogonalises outcome and treatment against X and minimises the R-loss.	Requires good nuisance estimates and careful cross-fitting; more sensitive to implementation error.	Advanced tier, if time allows.
DR-learner	Cross-fits a doubly-robust score across data partitions, so the estimate stays consistent if *either* the outcome model or the propensity model is correct.	Three-way cross-fitting costs sample efficiency and is easy to implement wrongly.	Advanced tier, alongside the R-learner.
Class transformation	Relabels the target so that a standard classifier's output maps onto uplift.	The simple form is only valid when treatment and control are balanced; the naive version applied to an 85/15 split is **wrong** (Section 7.3).	Only in its propensity-weighted form, and only with the trap documented.
Uplift trees / Uplift Random Forest	Trees split directly on a divergence between treated and control outcome distributions (KL, Euclidean, χ^2) rather than on outcome purity.	Fewer well-tuned implementations; can be slow at 14M rows and may need subsampling.	**Advanced-tier challenger.** A genuinely different model class, not a re-tuned version of the same one.
Causal forest	Recursive partitioning with honest estimation, producing valid confidence intervals for the effect.	Heavier; the honest-splitting sample cost bites on smaller data.	Advanced tier; valuable precisely because of the interval estimates.

Source: the meta-learner family (S/T/X) follows Künzel, Sekhon, Bickel & Yu, Metalearners for estimating heterogeneous treatment effects using machine learning, *PNAS 116(10), 2019; the R-learner follows Nie & Wager,* Quasi-oracle estimation of heterogeneous treatment effects*; uplift trees with KL / Euclidean / χ^2 split criteria and causal trees/forests (Athey & Imbens, PNAS 113(27), 2016) are as documented in the CausalML methodology reference.*

7.2 Why the X-learner is the primary candidate here

The published treatment ratio of Criteo-UPLIFT is **0.85**. The control group is therefore roughly one-sixth the size of the treated group, and it is the control group that carries the entire counterfactual. A T-learner fits its control model on that thin slice and then subtracts it from a much better-estimated treated model — so the noise in the smaller model dominates the difference. The X-learner was constructed for precisely this

asymmetry: it borrows strength from the larger group when imputing effects for the smaller one, and weights the two resulting effect models by the propensity score. That is a problem-driven model choice, which is the kind of justification a defence panel is actually looking for.

7.3 The class-transformation trap

The class-transformation method defines a transformed target $Z = 1$ when a customer is (treated *and* converted) or (control *and* did not convert), and $Z = 0$ otherwise. A standard classifier trained on Z then yields uplift as $2\,P(Z{=}1 \mid X) - 1$. That identity holds **only when treatment is assigned with probability** $1/2$. Applying it directly to a dataset with a treated share of 0.85 produces a systematically distorted uplift score that will still yield a plausible-looking Qini curve — which is what makes the error dangerous rather than obvious. If the method is used at all, it must be the propensity-weighted variant. Documenting this trap, and showing the biased-versus-corrected curves side by side, is a stronger result than quietly using the correct version.

7.4 Final recommendation

1. **Baselines (mandatory, first):** random targeting; blanket targeting; response model.
2. **MVP estimators:** T-learner and X-learner, both over LightGBM or XGBoost base learners.
3. **Advanced challengers:** Uplift Random Forest; a shared-representation neural model; optionally causal forest for interval estimates.
4. **Champion selection:** by Qini coefficient on the *validation* split with bootstrap intervals, confirmed once on the locked test split, and reported with the interval, never as a bare point estimate.

8 The Deep Learning Component

The track requires deep learning. Bolting an arbitrary neural network onto a tabular problem to satisfy that requirement is exactly the kind of decision this document is written to prevent. There is, however, a neural architecture family that exists *because of* this problem and is therefore a legitimate fit.

A shared-representation network learns a common encoding $\Phi(x)$ of the covariates and then branches into two outcome heads, one for the treated response and one for the control response, so that both heads share statistical strength in the encoder while remaining separate where they must be. The Dragonnet variant adds a third head that predicts the propensity score, on the argument that a representation sufficient for the propensity score is sufficient for the treatment-effect estimate.

Source: Shi, Blei & Veitch, Adapting Neural Networks for the Estimation of Treatment Effects, *NeurIPS 32, 2019 — the Dragonnet architecture and targeted regularisation.*

The honest framing for the defence. The neural estimator is included as a *challenger, not a favourite.* On tabular data of this kind, gradient-boosted trees are a strong opponent and may well win. Reporting that they did — with the comparison run properly — is a better outcome than a rigged comparison where the neural model wins because the baseline was undertrained. The deliverable here is the comparison, not a particular winner.

9 Baselines That Must Exist Before Any Uplift Model

Baseline	Definition	Why it is required
Random targeting	Select k customers uniformly at random.	The zero line of every Qini curve. Without it the word "uplift" has no referent.
Blanket targeting	Contact everyone.	Represents the do-nothing-clever policy the business may currently run; establishes the total available effect and the total cost.
Response model	Classifier for $P(Y{=}1 \mid X, T{=}1)$, ranked descending.	**The single most important comparison in the project.** This is what the company would build instead. If uplift targeting cannot beat it on incremental conversions at fixed budget, the project's central claim fails — and reporting that honestly is still a valid, defensible result.
Churn/propensity model	Optional, dataset-dependent.	Included where the outcome is retention rather than conversion, for the same reason as the response model.

Run the uplift-versus-response-model comparison by the analysis and design milestone (Week 12), not at the end. It is the experiment that tests the project's core hypothesis. If it comes out negative in Week 19 there is no time left to investigate why, and the team is left presenting a system whose premise it cannot support. If it comes out negative in Week 12, the investigation *becomes* the project's most interesting section.

10 Uplift Score to Decision: The Targeting Policy

A ranked score is not a decision. The decision layer converts $\hat{\tau}(x)$ into an action under a cost structure.

10.1 The economics

Let m be the margin realised on a conversion and c the cost of contacting one customer. The expected incremental profit of contacting a customer with estimated uplift $\hat{\tau}(x)$ is

$$\pi(x) \;=\; m\,\hat{\tau}(x) \;-\; c.$$

Contact is worthwhile when $\hat{\tau}(x) > c/m$. Note what this threshold is not: it is not "uplift > 0". A customer with a genuinely positive but tiny uplift is still a loss when the contact cost exceeds the expected margin gained. The break-even ratio c/m is a business input to the system, not a modelling choice, and the dashboard must expose it as a slider.

The decision layer and the simulator (Section 13) use the ITT-scale uplift $\hat{\tau}(x)$ throughout. The Wald estimate describes compliers only, at about 27.7 times ITT; it is reported for interpretation in Section 6 and never enters $m\,\hat{\tau}(x) - c$, where it would inflate the uplift term about 27.7-fold.

10.2 Policy modes

Mode	Rule
Fixed budget	Contact the top k by $\hat{\tau}(x)$, where k is set by the budget divided by c; suppress any selected customer whose $\hat{\tau}(x) \le 0$ even if it leaves budget unspent.
Profit-maximising	Contact every customer with $\hat{\tau}(x) > c/m$; budget becomes an output rather than an input.
Fixed target	Contact the smallest set expected to deliver N incremental conversions; reports the budget required and the uncertainty on it.
Suppression-only	Contact everyone *except* negative-uplift customers. A deliberately conservative mode, useful when a business will not accept reducing contact volume but will accept protecting revenue.

10.3 Negative uplift and the sleeping-dog rule

Any customer whose estimated uplift is negative *and* whose confidence interval lies entirely below zero is placed on a suppression list and is never contacted by the policy, regardless of remaining budget. This rule is not an optimisation. It is the one behaviour of the system that a conventional campaign engine cannot express at all, and it is the clearest demonstration in the whole project that the model is estimating an effect rather than a probability.

11 Uncertainty and Abstention

Every prior specification in this series required an abstention path, and it is more important here than in any of them, because the effect being estimated is small enough to be indistinguishable from noise in thin segments.

- Every reported uplift figure — for a customer, a decile, or a policy — carries a bootstrap confidence interval computed by resampling within treatment arms.
- Where a segment's interval straddles zero, the system returns `verdict = "insufficient_evidence"` rather than a directional recommendation. It does not round the estimate to a decision.
- Where a segment contains fewer than a configured minimum number of *control-group outcome events*, the system returns `verdict = "insufficient_control_data"`. This threshold is a configuration value, defended in the documentation, and logged with every response.
- The dashboard renders these two states visually distinct from a positive or negative verdict. An empty recommendation must look different from a confident one.

The reason this matters more than it looks: the top decile of a 14M-row dataset still contains only a few hundred control-group conversions (Section 17). A point estimate computed from a few hundred events, reported without an interval, is the most likely single source of a false claim in this entire project.

12 Model Output Schema

Frozen early, and treated as a contract between the modelling members and the backend member.

```
{
  "customer_id": "c_00193472",
  "model_version": "netlift-xlearner-2026.11.03",
  "uplift_score": 0.0041,
  "uplift_ci_95": [0.0009, 0.0074],
  "decile": 1,
  "segment_label": "persuadable",       // persuadable | sure_thing |
                                        // lost_cause | sleeping_dog | unknown
  "expected_incremental_profit": 1.87,  // m * uplift - c, in currency units
  "recommendation": "contact",          // contact | suppress | no_action
  "verdict": "confident",               // confident | insufficient_evidence |
                                        //             insufficient_control_data
  "top_drivers": [
      {"feature": "f3",  "contribution":  0.0012},
      {"feature": "f7",  "contribution": -0.0005}
  ],
  "scored_at": "2026-11-03T14:22:10Z"
}
```

Two design notes. First, `segment_label` is a derived, presentational field and must never be treated as ground truth — no dataset labels customers as persuadables, and the label is an inference from the estimated effect plus the predicted baseline conversion probability. Second, `recommendation` and `verdict` are separate fields on purpose: a system that collapses "do not contact" and "we do not know" into one value has thrown away the distinction the abstention logic exists to preserve.

13 Campaign Policy Simulator

The single component that makes this project legible to a non-technical evaluator in ten seconds.

Given a held-out randomized test set, a per-contact cost, a margin, and a budget, the simulator replays each candidate policy over the test set and reports, for each: number contacted, total cost, incremental conversions with a confidence interval, incremental profit, and the profit per unit spent. The output is one table and one chart in which the uplift policy, the response model, blanket contact and random contact appear side by side.

This is the artefact to open the final presentation with, and the one to put at the top of the README. It converts a Qini coefficient — which means nothing outside the field — into a currency figure with an error bar, which means something to everyone.

13.1 How to state the simulator's output without overclaiming

The simulator produces an *estimated* incremental profit under *assumed* cost and margin inputs, computed over held-out experimental data. It is not realised money. The difference is small in wording and large in credibility, and it is the fastest way for an otherwise strong project to lose an examiner's trust.

Do not write	Write instead
"Saved the company $500,000."	"Estimated a $500,000 incremental-profit opportunity at the stated contact cost and margin, measured on held-out experimental data."
"Improved retention by 12%."	"Produced 12% more incremental conversions than the response-model baseline at an equal budget (95% CI: 6%–18%)."
"Our model is 94% accurate."	"Accuracy is undefined for this target; Qini coefficient 0.31 (95% CI: 0.24–0.38) on the locked test split."
"Identified the sleeping dogs."	"Estimated a confidently negative treatment effect for this subgroup; no dataset labels individuals as such."

The right-hand column is longer and it is what a senior reviewer reads as competence rather than hedging. Apply the same rule in the final report, the presentation, the README and any CV line derived from this project.

14 Simulated Campaign Replay

Read this before the rest of the section, because the honest framing matters more than the design. Criteo-UPLIFT and Hillstrom are **fixed historical datasets**. There is no live audience to score, no target list that will be handed to a campaign system, and no outcome window that will close while the team waits. Every outcome in both datasets was recorded years ago and is already in the file.
What this project builds is therefore a **simulated campaign replay**: a slice of the held-out test split is treated as "next month's campaign", the system makes its decisions on it without looking at the outcomes, and the outcomes are then revealed and compared against what was predicted. That is a real and measurable exercise — it is simply not a live campaign, and this document does not claim it is. Section 15 states separately what a genuine deployment would additionally require.

14.1 Why the replay is worth building anyway

A project that stops at "here is a Qini curve" has never tested whether its scores translate into a decision that would have paid off. The replay does exactly that, end to end, and it is possible precisely *because* the data is historical: the counterfactual outcomes are already recorded, so the comparison can be made immediately instead of after a waiting period.

14.2 The replay loop

```
[1] DEFINE THE SCENARIO   offer cost per contact, margin, budget, and the slice of
          |               the held-out test split that stands in for the audience
          v
[2] SCORE                 champion model scores that slice -> uplift + interval
          |               (outcomes are NOT read at this stage)
          v
[3] ALLOCATE              decision layer selects targets under the budget;
          |               suppresses confidently negative uplift; abstains where thin
          v
[4] HOLD OUT              a randomized slice of the SELECTED targets is marked
          |               as not-contacted -- this is what makes step [6] possible
          v
[5] REVEAL OUTCOMES       the recorded outcomes for that slice are now read
          |
          v
[6] MEASURE               measured incremental effect = treated rate - holdout rate,
          |               with a bootstrap confidence interval
          v
[7] COMPARE               predicted incremental conversions vs measured, plus the
          |               same comparison for the response-model and random policies
          v
[8] RECORD                calibration record; the whole run is logged to MLflow
```

The one discipline that makes the replay honest: steps [1]–[4] must not touch the outcome column. Implement it so that they cannot — the scoring and allocation functions receive a frame without the outcome column at all, and a test asserts that. If the allocation can see the answers, the replay proves nothing.

14.3 What the replay produces

For each scenario, one table: predicted incremental conversions against measured incremental conversions, each with an interval, for the uplift policy, the response-model policy, blanket contact and random contact at an identical budget.

This is the strongest single artefact the project can put in front of an evaluator, and a Qini coefficient alone cannot produce it: it answers "would this decision have paid off?" rather than "is the ranking good?". It costs about a day to build on top of the evaluation harness that already has to exist.

14.4 Where this shows up in the workload

The replay adds: scenario and assignment records in the database and the ERD; holdout-selection logic; the outcome-reveal boundary and the test that enforces it; the measurement job; and the scenario state machine (`defined` → `scored` → `allocated` → `revealed` → `measured`), which is also the state diagram the DEPI documentation asks for (Section 45).

15 What a Real Deployment Would Additionally Require

Nothing in this section is built or exercised this semester. It is recorded because the difference between the replay above and a live system is a design question the team should be able to answer, and because it is the first question a technically strong examiner asks. Presenting it as project work would be dishonest; leaving it out would look like the team had not thought about it.

What changes	Why
A permanent randomized holdout	In production the counterfactual is not in a file — it has to be manufactured, by permanently withholding contact from a small randomized share of the addressable base. Without it, incremental effect stops being measurable the moment the system goes live.
A real outcome window	Conversions arrive days to weeks after the decision, so the true effect signal is delayed and only ever available in aggregate. Feature and score drift are immediate; effect drift is not.
Treatment assignment as a record	Who was assigned to which arm, written at assignment time, is what makes a campaign auditable after the fact.
A retraining rule that is not purely reactive	Because the true signal is slow, retraining combines a scheduled cadence with drift triggers rather than waiting for measured decay.
Compliance and contamination handling	Assigned ≠ received once a real channel is involved (Section 6).

The sentence to use if asked. "Our data is historical, so we replay campaigns over a held-out split rather than running them. The replay measures the decision, not just the ranking. A live deployment would need a permanently maintained randomized holdout to keep that measurement possible, and we designed for that — but we did not exercise it, because the data does not allow it."

Part III

Data Strategy

16 Dataset Strategy

Every figure in this section is taken from the dataset's own published documentation and is cited where it appears. Nothing here is recalled from memory. Figures that are arithmetic consequences of the published ones are labelled *derived* and should be re-checked by the team against the downloaded files in Week 9–10, because a specification is not a substitute for opening the data.

16.1 Criteo-UPLIFT — the primary dataset

Property	Value
Rows	13,979,592
Columns	16: twelve features and four non-feature columns
Features	12, named `f0`–`f11`, all float, anonymised
Non-feature columns	`treatment` (the randomized assignment), `visit` and `conversion` (the two candidate outcomes), and `exposure`
Treatment ratio	0.85 published; 11,882,655 of 13,979,592 rows treated, counted
Average visit rate	0.046992
Average conversion rate	0.00292
Targets available	`visit` (binary) or `conversion` (binary). Whichever is not the target is itself post-treatment and is leakage (Section 18)
`exposure`	Post-treatment. **Forbidden** as a feature *and* as a treatment variable; rejected by name in `validate_features()` and excluded from the balance check (Sections 6 and 18)
Design	Large-scale randomized advertising experiment

Source: dataset description as published in the `scikit-uplift` documentation for `fetch_criteo`. The dataset was released with Diemert, Betlei, Renaudin & Amini, A Large Scale Benchmark for Uplift Modeling, *AdKDD &*

TargetAd Workshop, KDD 2018.

What makes it the right primary choice: it is a genuine randomized experiment at a scale where the statistical questions in this project become real rather than illustrative, and it is the standard public benchmark in the field, so the team's Qini numbers sit in a context others can recognise.

What makes it hard, and why that is the point: the features are anonymised floats. There is no domain interpretation to lean on, no feature engineering shortcut, and no way to inspect whether a feature is post-treatment except by reasoning about the experiment's design. Everything the model achieves has to come from methodology.

16.2 Hillstrom MineThatData — the second dataset

Property	Value
Customers	64,000, each having last purchased within twelve months
Arms	Three, randomly assigned in equal thirds: Mens E-Mail, Womens E-Mail, No E-Mail (control)
Features	`Recency`, `History_Segment`, `History`, `Mens`, `Womens`, `Zip_Code` (Urban/Suburban/Rural), `Newbie`, `Channel`
Outcomes (2 weeks after)	`Visit` (binary), `Conversion` (binary), `Spend` (continuous currency)

Source: variable list and the equal-thirds randomization as published by Kevin Hillstrom in the MineThatData E-Mail Analytics and Data Mining Challenge announcement (`blog.minethatdata.com`, March 2008), cross-checked against the `scikit-uplift fetch_hillstrom` documentation.

Its three distinct jobs in this project:

1. **Pipeline sanity check.** It is small and its features are human-readable, so every component — the balance check, the split, the Qini harness, the simulator — can be developed and debugged here in seconds rather than minutes, before being pointed at 14M rows.
2. **Multi-treatment.** It has *two* distinct treatments. Pooling them into a single "emailed" group estimates the average effect of two different interventions, which is a real methodological error with a clean demonstration (Section 21).
3. **Revenue uplift.** `Spend` gives a continuous outcome, so the Advanced tier can estimate uplift in currency rather than in probability — which is what a business ultimately wants and what makes the simulator's profit figure directly estimable rather than derived through an assumed margin.

16.3 Comparison

	Criteo-UPLIFT	Hillstrom
Scale	13.98M rows	64K rows
Arms	1 treatment, 1 control	2 treatments, 1 control
Balance	85% treated	Equal thirds
Feature meaning	Anonymised	Interpretable
Continuous outcome	No	Yes (`Spend`)
Role	Primary result	Development, multi-arm, revenue uplift

17 The Central Data Challenge: A Thin Counterfactual

This section contains the number the team should keep in front of them for the whole semester.

Counted from the file: **11,882,655 treated** and **2,096,937 control** rows. The control arm converts at 0.1938%: **4,063 control conversions** in total, counted. That is below the 6,100 the published all-rows average of 0.00292 would suggest, because a positive treatment effect pulls that average above the control rate — so using the published average here would have overstated the counterfactual evidence by half.

Source: counted by `scripts/criteo_complier_baseline.py`; see also `reports/t16_dataset_verification.md`.

Now divide the scored population into deciles. Each decile holds about 1.40M rows, of which roughly 210,000 are control. At the measured control rate that is about **406 control conversions** per decile if they spread evenly, and fewer in the deciles that matter most.

Every uplift number this project reports for a decile rests on a few hundred control-group events. That is why:

- confidence intervals are mandatory rather than a nice extra (Section 28);
- the train/validation/test split must be stratified jointly on treatment *and* outcome (Section 19), or a fold can end up with too few control conversions to estimate anything;
- "the model found a strong effect in the top decile" is a claim that requires an interval before it is spoken aloud;
- `visit` (rate 0.046992, roughly sixteen times more frequent than conversion) is the sensible development target while the pipeline is being built, with `conversion` as the headline business target.

18 Leakage and Validity Audit

Data leakage in a causal project has a different shape from leakage in a classification project, and a team that only knows the classification version will miss it.

Leakage type	Rule
Post-treatment features	Any feature that could have been affected by the treatment must be removed. Using a variable measured after the intervention to predict the effect of the intervention guarantees an inflated, meaningless result. In Criteo this cannot be checked by inspection because the features are anonymised, so the project relies on the dataset's stated design and records that reliance as an assumption.
Outcome-as-feature	In Hillstrom, `Visit`, `Conversion` and `Spend` are all post-treatment outcomes. Using `Spend` or `Visit` as an input when the target is `Conversion` is leakage, not feature engineering — and it is an easy mistake because they sit in the same table as the features. Criteo carries the same trap: `visit` and `conversion` are both post-treatment, and whichever one is not the target is forbidden as an input. `validate_features()` rejects it by name, alongside `exposure`.
Treatment-indicator leakage	Outside the S-learner, where it is the design, the treatment flag must never enter the feature matrix.
Criteo `exposure`	**Forbidden, both as a feature and as a treatment variable.** It is post-treatment: it records whether an assigned user was actually shown the advertisement (Section 6). As a feature it leaks the treatment's own consequence. More dangerously, it must *not* be substituted for `treatment` as the treatment variable in an uplift model: exposure among the treated is self-selected rather than randomized, so modelling uplift on it discards the randomization the entire project depends on. `validate_features()` rejects this column by name.
Preprocessing across the split	Scalers, encoders, imputers and target-encoded categoricals are fitted on the training fold only and applied to the others.
Metric overfitting	Hyperparameters are selected on validation, never on the test split. The test split is written down as locked and is touched exactly once, at champion confirmation.
Duplicate rows	Checked for and documented. Duplicates that straddle a split boundary inflate every metric.

Implement this as an automated check, not a habit: a `validate_features()` function that fails the pipeline if a forbidden column reaches the model matrix, and a unit test that asserts it fails. A guarantee enforced by a test is worth more in a defence than a guarantee described in a document.

19 Splitting Strategy

The rule: stratify jointly on (treatment, outcome). A plain random split, and even a split stratified on the outcome alone, can produce a fold whose control arm contains too few conversions to estimate uplift at all. Joint stratification preserves both the treated/control proportion and the event rate within each arm in every fold. Verify it after splitting rather than trusting it: assert the four cell counts per fold and log them.

- **Split ratio:** 60 / 20 / 20 train / validation / locked test, with a fixed seed recorded in configuration and logged to MLflow.
- **Cross-validation:** stratified K-fold on the same joint key, used for model selection inside the training split.
- **Cross-fitting:** the X- and R-learners require their nuisance models to be fitted out-of-fold. Implementing this incorrectly reintroduces exactly the bias the estimator exists to remove, and the error is invisible in the metrics. Use the library implementation rather than a hand-rolled loop, and write a test that checks fold membership.
- **No entity duplication.** Unlike the earlier projects in this series, neither dataset has repeated entities across rows, so group-based splitting is not required — but this must be *verified*, not assumed, by checking for duplicate feature vectors.

20 Randomization Integrity Check

Before a single model is trained, the project verifies that the experiment it depends on actually behaves like a randomized experiment.

1. For every covariate, compute the standardized mean difference between treated and control. **Criteo's `exposure` column is excluded from this check.** It is zero for every control row by construction, so its standardized mean difference is enormous by design. Including it would report a catastrophic randomization failure that is an artefact of the check, not a finding about the data. It is post-treatment and therefore not a covariate at all.
2. Flag any covariate whose absolute SMD exceeds a pre-registered threshold (0.1 is the conventional choice; the threshold is recorded in config, not chosen after seeing the results).
3. Fit a classifier that tries to predict the treatment flag from the covariates alone. Under correct randomization it should be unable to: an ROC–AUC materially above 0.5 indicates that treatment is predictable from covariates and the unconfoundedness assumption is in trouble.
4. Verify overlap: check that control rows exist across the covariate space, not only in part of it.
5. Record all of this as a dated report artefact, not as a notebook cell.

This costs half a day and is the highest-value half-day in the project. It is the step that lets the team answer "how do you know your causal assumption holds?" with evidence instead of with a citation. Almost no student project does it.

21 Multi-Treatment Handling

Hillstrom has two treatment arms, not one. Collapsing *Mens E-Mail* and *Womens E-Mail* into a single "emailed" group and comparing against control estimates the average effect of two different interventions on a mixed population — a quantity that corresponds to no decision anyone can take. The correct options are: (a) analyse one arm at a time against the shared control, which is the MVP approach; or (b) model the arms jointly and select, per customer, the arm with the highest estimated uplift, which is the Advanced-tier approach and is the more interesting of the two.

Option (b) is a genuine extension: it turns "should we contact?" into "which offer should this customer receive?", which is the form the question takes in every real marketing system.

22 Choosing the Outcome Variable

The outcome is a decision, not a given, and the wrong choice produces a technically correct system that optimises the wrong thing.

Outcome	Trade-off
`visit` (Criteo)	Sixteen times more frequent than conversion, so estimates are far more stable. Weakly tied to money. **Use it to develop and debug**, and report it alongside the headline.
`conversion` (both)	The headline business outcome, and the one with the thin-counterfactual problem of Section 17.
`spend` (Hillstrom)	Continuous, and closest to what the business cares about. Uplift in currency is directly usable by the decision layer without assuming a margin. Higher variance, and sensitive to a handful of large purchases — so report a trimmed or winsorised variant alongside the raw one and say which is which.

A high-uplift customer is not always a high-value customer. A campaign can raise retention by ten percent among customers who each contribute very little, and produce almost no additional revenue. This is why the decision layer optimises $m\,\hat{\tau}(x) - c$ rather than $\hat{\tau}(x)$ alone (Section 10), and why revenue uplift on Hillstrom's `spend` column is worth reaching in the Advanced tier: it removes the assumed margin from the calculation and estimates the money directly. Ranking by uplift and ranking by incremental profit are different orderings, and the second one is the product.

23 Complete Data Pipeline

```
raw download (versioned, checksummed, never edited in place)
      |
      v
schema validation  -- column names, dtypes, ranges, null policy
      |
      v
duplicate detection and removal (documented count)
      |
      v
randomization integrity report  -- SMD table, treatment-predictability AUC
      |
      v
leakage audit  -- forbidden-column check (automated, test-enforced)
      |
      v
joint stratified split (treatment x outcome), seed logged
      |
      v
preprocessing FITTED ON TRAIN ONLY
(impute -> encode -> scale; artefacts serialised and versioned)
      |
      v
feature matrix + treatment vector + outcome vector, per fold
      |
      v
model training (baselines first)  ->  MLflow run per configuration
      |
      v
evaluation harness (Qini, AUUC, uplift@k, bootstrap CIs)
      |
      v
champion selection on validation  ->  confirm ONCE on locked test
      |
      v
MLflow Model Registry  ->  scoring service
```

24 Data Quality and Documentation Deliverables

- A dataset card for each source: origin, licence terms as published on the download page, size, arms, outcome definitions, and known limitations.
- The randomization integrity report (Section 20).
- A cleaning log recording every row removed and why, with counts.
- A data dictionary for the engineered feature set.
- An explicit statement, in the final report, of which figures were verified against the downloaded files and which were taken from documentation.

Verify the unit of observation before quoting the row count. "13,979,592 rows" is a row count, and a row count is not automatically a count of independent customers. Before the number appears in a report or on a CV, the team must confirm from the dataset's own documentation and from the file itself what one row represents — one user, one impression, one user–campaign pair — and whether any identifier repeats. This matters for two reasons beyond honesty: repeated units across a split boundary are leakage, and standard errors computed as if correlated rows were independent are too narrow, which would make every confidence interval in the project optimistic. If the unit turns out to be anything other than an independent customer, the split becomes group-based on that unit and the report says so. Record the finding either way — "we checked, and here is what a row is" is a strong answer to a question most teams cannot answer at all.

Licensing. Both datasets are public and widely used for research, but the team must read the licence text on each download page before redistributing any part of the data in the project repository, and must record what it says. Do not commit raw data to the repository; commit a loader script, a checksum, and a small sample.

Part IV

Engineering

25 Training Strategy

1. **Develop on Hillstrom, report on Criteo.** Every component is built and debugged against 64K rows before it is pointed at 14M. This is a schedule decision as much as a technical one: it removes the long feedback loop from the phase where mistakes are most likely.
2. **Develop against `visit`, report on `conversion`.** The visit rate is roughly sixteen times the conversion rate, so curves are stable enough to debug against. The headline result is still reported on `conversion`, and both are reported in the appendix.
3. **Baselines before estimators.** Non-negotiable, per Section 9.
4. **One configuration file per experiment.** No hyperparameter is ever typed into a notebook cell. Every run reproducible from `config/*.yaml` plus a git commit hash.
5. **Sampling for iteration, full data for reporting.** A stratified 10% subsample of Criteo for iteration; every reported number computed on the full data.
6. **Class imbalance.** With a 0.29% positive rate, resampling or class weights change the base-rate calibration of both arms — which changes their difference. Any imbalance handling must be applied *identically* to the treated and control models and its effect on the uplift estimate checked explicitly, not assumed to cancel.

26 Hyperparameter Tuning Under an Uplift Objective

Standard tuning tooling optimises a supervised loss on an observed label. There is no observed label here. Tuning a T-learner's two base models to maximise their individual ROC–AUC optimises each model's ability to predict conversion — which is not the same as, and can actively conflict with, the accuracy of their *difference*. Two well-calibrated but individually mediocre models can produce better uplift estimates than two individually excellent ones.

The required approach: the objective for hyperparameter search is the Qini coefficient computed on out-of-fold predictions inside the training split, using stratified K-fold on the joint (treatment, outcome) key. Search with Optuna or a comparable optimiser; log every trial to MLflow; select on validation; confirm once on the locked test split.

Report the tuning honestly: include, in the final report, how many trials were run for each model family. A champion selected after 200 trials compared against a baseline given 5 is not a comparison, and an evaluator who asks this question and gets a straight answer will trust the rest of the report considerably more.

27 Evaluation Metrics

27.1 Why the usual metrics do not apply

Accuracy, precision, recall, F1 and ROC–AUC all require a per-row ground-truth label. The label here, τ_i, does not exist for any row (Section 2.2). These metrics are therefore not merely weak in this setting — they are undefined for the quantity being predicted. They remain valid for the *baseline response model*, because that model predicts an observed outcome, and reporting them there while explaining why they are absent for the uplift models is a useful contrast to include.

27.2 The metric set

Metric	What it is and why it is included
Uplift curve	Population sorted descending by predicted uplift; the curve plots cumulative incremental outcomes against the number of customers targeted. It answers the operational question directly: if we contact the top n, how many extra conversions do we get?
Qini curve and Qini coefficient	The Qini curve is the uplift curve's classical form; the Qini coefficient normalises the area between the model's curve and the random-targeting diagonal against the area of the optimum curve, summarising the curve as one number.
AUUC	Area under the uplift curve; reported alongside Qini because the two normalise differently and the field is not fully consistent about which is meant by which name. Report both, and state the implementation used.
uplift@k	Uplift measured on the top k fraction, which is the quantity a fixed budget actually buys. Note that the `overall` and `by_group` computation strategies give different numbers; state which was used.
Cumulative gain	Companion visualisation for the dashboard.
Incremental profit	The business-facing metric from the simulator (Section 13): $m\times$ incremental conversions − contact cost.

Source: the Qini coefficient as the normalised area under the Qini curve, and the two uplift@k strategies, are as defined in the ***`scikit-uplift`*** *metrics documentation, which cites Radcliffe,* Using control groups to target on predicted lift: building and assessing uplift models, *Direct Marketing Analytics Journal 3:14–21, 2007.*

Report the implementation, not just the number. Qini and AUUC differ across libraries in normalisation and in whether negative effects are included in the optimum curve. A Qini coefficient quoted without naming the library and the settings is not reproducible. Name them in the report and pin the library version in `requirements.txt`.

28 Statistical Significance — The Section That Protects Every Claim

A Qini coefficient is a statistic computed from a finite sample. On a dataset whose deciles contain a few hundred control conversions (Section 17), two models can differ in Qini purely by chance. Comparing point estimates and declaring a winner is the most likely way this project produces a false result, and it is exactly the claim a sharp examiner will probe.

Required protocol:

1. **Bootstrap.** Resample the evaluation set with replacement *within each treatment arm* (preserving the arm proportions), recompute the metric on each resample, and report the 2.5th and 97.5th percentiles as a 95% interval. Use at least 1,000 resamples; state the number.
2. **Model comparison.** Compare champion and challenger on the *same* bootstrap resamples and report the distribution of the difference. If that distribution straddles zero, the correct conclusion is that the models are not distinguishable on this data — and that is a finding, not a failure.
3. **Curve bands.** Plot Qini curves with bootstrap confidence bands, not as bare lines. A curve with a visible band is self-documenting.
4. **A negative control.** Fit the entire pipeline on a randomly permuted treatment indicator. The measured uplift must collapse to approximately zero. If it does not, there is a bug or a leak, and this test will find it when nothing else does.

The permuted-treatment negative control is the single cheapest, highest-value test in this document. It takes an hour to implement, runs automatically, and catches the class of error that otherwise surfaces only during the defence.

29 Error Analysis and Segment Audit

- **Decile table.** For each predicted-uplift decile: number treated, number control, outcome rate in each arm, observed uplift, predicted uplift, and the interval. The diagonal of this table is the model's calibration story.
- **Uplift calibration plot.** Predicted uplift against observed uplift by bin. This is the uplift analogue of a reliability diagram and it is more informative than any single coefficient.
- **Negative-uplift segment profile.** Who are the sleeping dogs? Describe them in the dataset's own feature terms (feasible on Hillstrom; on Criteo, describable only in terms of anonymised features, and the report should say so).
- **Where the model fails.** Segments where the predicted and observed uplift disagree most, and a stated hypothesis for each.

30 Behavioural and Invariance Tests

Tests that check the model behaves like an effect estimator rather than a relabelled classifier.

1. **Permuted treatment $\Rightarrow$ zero uplift.** As above. The primary invariant.
2. **Rank stability under reseeding.** Retrain with a different seed; the top-decile membership should overlap substantially. Violent instability means the model is fitting noise.
3. **Uplift $\neq$ response ordering.** Correlate the uplift ranking with the response-model ranking. If they are nearly identical, the uplift model has learned the conversion probability, not the effect — which is the S-learner failure mode made visible.
4. **Monotone budget response.** Incremental conversions must not decrease as the targeted fraction increases in the cumulative curve. A violation indicates a bug in the curve computation.
5. **Subgroup sanity on Hillstrom.** The Mens-email arm should show its strongest effect among customers with a history of mens purchases. If it does not, investigate before proceeding — this is a domain check that Criteo's anonymised features cannot provide, and it is one of the reasons Hillstrom is in the project.

31 Explainability

- **SHAP on the uplift estimate**, not merely on the outcome models, so the attribution answers "why is this customer persuadable?" rather than "why is this customer likely to buy?".
- **Segment profiling**: a plain-language description of the top-uplift decile expressed in feature terms, which is what a marketing user will actually read.
- **Decile table** (above) as the model-level explanation.
- **Per-customer drivers** surfaced in the API response (Section 12).

State the limit of the explanation. SHAP attributes the model's *score* to input features. It does not establish that those features cause the effect. The correct sentence in the report is "these features drive the model's uplift prediction", never "these features cause customers to be persuadable". Getting this distinction right in a causal project signals more maturity than any modelling choice in the document.

32 Concept Drift and Monitoring

Uplift models decay faster than outcome models, because they estimate a response to an intervention and audiences habituate to interventions. An offer that produced real lift last quarter can produce none this quarter with no change in the customer base at all.

Monitor	Detail
Feature drift	Population Stability Index or KS statistic per feature, scored batch against training reference; alert thresholds set in config.
Score drift	Distribution of predicted uplift over time; a collapsing spread is an early warning.
Effect drift	The one that matters: measured uplift in the production holdout, tracked over time with its interval.
Data quality	Null rates, range violations, schema changes, blocking the scoring run on failure.
Retraining trigger	Documented rule combining a drift threshold with a scheduled cadence; every retrain logged as a new registered model version with its evaluation attached.

32.1 Delayed feedback — a deployment problem, not one this project faces

This subsection describes deployment, not this semester's work. On the datasets used here every outcome is already recorded, so nothing is ever waited for: the replay of Section 14 measures effect immediately. The paragraph below applies the day the system is pointed at a live campaign, and is written so the team can answer the question rather than because it is a risk they are carrying.

In a live system the quantity being monitored is a *difference between two group rates*, and it cannot be computed until a campaign's outcome window has closed — days to weeks after the decision, and only in aggregate. There is no per-prediction feedback signal. A model can therefore degrade for a full campaign cycle before anything measurable moves. The response is layered:

Signal	Latency in deployment	What it tells you
Feature drift (PSI/KS)	Immediate	The population has moved. A leading indicator only — it does not prove the effect has changed.
Score distribution drift	Immediate	The model is behaving differently on new data.
Early-proxy outcome	Short	Where a fast, correlated outcome exists (a visit rather than a purchase), it gives a directional read long before the real one. Explicitly a proxy; label it as one.
Measured effect in the permanent holdout	**One outcome window**	The only true signal. Nothing else substitutes for it.
Predicted-versus-measured calibration	One outcome window	Whether the model's magnitudes, not just its ordering, still hold.

What the team actually builds from this: the two immediate monitors (feature drift and score drift), which are computable on historical data by splitting it by time or by simulating a shifted population, plus the monitoring tab that displays them. The two delayed signals are *designed and documented, not implemented*, and the report says so.

The permanent randomized holdout — a deployment requirement, not a project deliverable. In production a small randomized control group must be maintained *forever* — typically a few percent of the addressable base, withheld from all campaigns — because it is the only mechanism by which incremental effect can continue to be measured once the system is live. On this project the counterfactual comes from the dataset instead, so there is nothing to withhold. The holdout appears in the architecture diagram marked as deployment-only, and the team should be able to defend its cost: the revenue forgone by not contacting that slice is the price of knowing whether contacting anyone works at all.

33 Experiment Tracking and Reproducibility

- **MLflow tracking** for every run: parameters, the config file, the git commit, dataset version and checksum, the split seed, all metrics with their intervals, and the Qini/uplift curves as artefacts.
- **MLflow Model Registry** with explicit stages, so that the model behind the API is always identifiable by version.
- **Champion–challenger record**: a maintained table of every model tried, its metrics with intervals, and a one-line reason it did or did not become champion. This table is a deliverable, not a working note.
- **Determinism**: seeds fixed and logged; environment pinned; one command reproduces any reported number from a clean checkout.

34 Testing

Layer	Coverage
Unit	Metric implementations verified against hand-computed values on a tiny fixture; split function; decision-threshold logic; schema validation.
Data	Forbidden-column check; joint-stratification assertion; duplicate check; null and range policy. Failures block the pipeline.
Model	The behavioural and invariance tests above, run as automated tests rather than as one-off notebook checks.
Integration	End-to-end run on the Hillstrom fixture from raw file to scored output.
API	Contract tests on every endpoint, including the two abstention verdicts and malformed input.
CI	GitHub Actions running lint, type check, unit, data and integration tests on every pull request.

Writing the Qini implementation test against a hand-computed fixture is worth doing even though a library is being used. It forces at least one team member to understand the metric arithmetically, and that person will answer the defence question about it fluently.

35 Deployment Architecture

35.1 API design

```
POST /score           single customer  -> full response object (S 12)
POST /score/batch     CSV or JSON array -> scored file + summary
POST /policy/simulate budget, cost, margin -> policy comparison table
GET  /model/info      registered version, training date, metrics + CIs
GET  /monitor/drift   latest drift report
```

```
GET  /health          liveness / readiness
```

35.2 Serving stack

Layer	Choice
Service	FastAPI with Pydantic request/response models, auto-generated OpenAPI documentation.
Model loading	Pulled from the MLflow Model Registry by version at startup; version returned in every response.
Batch path	The primary production mode — campaign lists are scored in bulk on a schedule, not one customer at a time.
Real-time path	Retained for the demo and for on-the-fly decisions in a servicing channel.
Container	Docker, with a pinned base image and a slim runtime layer.
Cloud	Azure: Blob Storage for datasets and artefacts, Azure ML for the managed endpoint and registry integration, Container Registry for images, Application Insights for logs and latency.
Frontend	Streamlit for the dashboard and simulator.
CI/CD	GitHub Actions: test, build image, push, deploy.

The API-first note, restated where it belongs. `/score/batch` and `/policy/simulate` are defined as a contract independent of the Streamlit UI. If a web team is ever added, their backend becomes a client of this service and nothing on the ML side changes. Designing it this way costs nothing today; retrofitting it later costs a rewrite.

Azure cost discipline. Compute instances continue to bill while idle. Shut them down after every session, set a budget alert well below the available credit, and train locally — the project's models are gradient-boosted trees on tabular data and do not need cloud GPUs. Use the credit for the parts of the curriculum that require it: the managed endpoint, storage, monitoring and the registry.

36 Technology Stack

Layer	Choice	Reason
Language	Python 3.12	Track standard; the entire causal-inference ecosystem lives here. Pinned to 3.12 for the reason in the note below.
Data	pandas, NumPy, PyArrow / Parquet	14M rows is comfortable in memory in Parquet; Polars is a reasonable substitute if memory becomes tight.
Classical ML	scikit-learn, LightGBM, XGBoost	Strong tabular baselines and fast enough to tune properly.
Uplift	`scikit-uplift`, `causalml`	`scikit-uplift` for the metric harness and the meta-learner basics; `causalml` for uplift forests and the wider meta-learner set.
Deep learning	PyTorch	The shared-representation estimator (Section 8).
Explainability	SHAP	Standard, and integrates with the tree models.
Tuning	Optuna	Qini-objective search with trial logging.
Tracking	MLflow	Required by the track and genuinely the right tool here.
Statistics	SciPy, statsmodels	Bootstrap intervals, balance tests.
Viz	Matplotlib, Plotly	Static report figures and interactive dashboard charts.
API	FastAPI, Uvicorn, Pydantic	Typed contracts and free OpenAPI docs.
Frontend	Streamlit	The audience is analytical; a React frontend would add build complexity without adding evidence of ML skill.
Storage	PostgreSQL, Azure Blob	Scoring logs and campaign metadata; datasets and artefacts.
Container	Docker	Reproducible runtime.
Cloud	Azure ML, ACR, App Insights	Track alignment and a real deployment story.
CI/CD	GitHub Actions	Free for the repository and enough for this pipeline.
Docs	MkDocs or Sphinx	Module-level documentation as required by the project standards.

Two environment facts that are easy to get wrong. Both were verified against PyPI, python.org and a working Windows install on 21 September 2026.
`causalml` contains compiled extension modules and publishes prebuilt Windows wheels for cp311 and cp312, but not for cp313. On a Python version with no wheel, `pip` falls back to building from source, which requires a C++ toolchain and frequently fails. Separately, python.org stops publishing Windows installers for a release line long before that line stops receiving security fixes: the newest Windows installer for 3.11 is 3.11.9 from April 2024, against 3.12.10 from April 2025. Python 3.12 is the only line that has both a `causalml` wheel and a recent installer, which is why the project pins it.
MLflow 3.x **refuses the plain `./mlruns` directory store** and raises on first use. The tracking backend must be a database URI. The project uses `sqlite:///mlflow.db` locally.

37 Repository Structure

A flat notebook folder is the most common way a strong project loses marks on software-engineering criteria. The layout below is the one the code standards in this document imply.

```
netlift/
  README.md                  results summary, install, config, run, API quickstart
  pyproject.toml             pinned dependencies
  Dockerfile  docker-compose.yml
  .github/workflows/ci.yml   lint, types, unit, data, integration tests
  config/
    data.yaml  model.yaml  policy.yaml  paths.yaml
  src/netlift/
```

```
  data/       loaders.py  schema.py  audit.py  split.py  preprocess.py
  features/   engineering.py  selection.py
  models/     baselines.py  meta_learners.py  forests.py  neural.py  registry.py
  evaluation/ metrics.py  bootstrap.py  curves.py  negative_control.py
              behaviour_tests.py  error_analysis.py
  decision/   policy.py  simulator.py  abstention.py
  api/        service.py  schemas.py  monitoring.py
  utils/      logging.py  config.py  seeds.py
  explain/    shap_uplift.py  segments.py        added in milestone 5
  dashboard/  app.py  tabs/                      added in milestone 5
tests/        unit/  data/  model/  integration/  api/
notebooks/    exploration only; nothing here is a deliverable
docs/         specification, milestone documents, diagrams, dataset cards,
              data dictionary, test plan, user manual
reports/      generated: balance report, decile tables, figures
```

Two rules that matter more than the layout itself: nothing in `notebooks/` is imported by anything in `src/`, and every number that appears in the final report is produced by a script in `src/` writing into `reports/`, never copied by hand out of a notebook cell.

38 Hardware

Training is gradient boosting over tabular data. A laptop with 16 GB of RAM is sufficient; the RTX 4060's 8 GB of VRAM is used only by the Advanced-tier neural estimator, which is a small multi-layer network and fits comfortably. The real resource constraint is memory during preprocessing of the full Criteo file, which is managed by loading Parquet column subsets and by processing in chunks. No cloud GPU is required at any point, and requesting one would be a signal that the team has not understood its own workload.

39 Curriculum Coverage — Stated Honestly

Track component	Covered	Where
Python	Yes	Throughout; packaged, typed, tested modules.
Data preprocessing	Yes	Part III pipeline, leakage audit, split discipline.
Data visualization	Yes	Qini curves with bands, calibration plots, decile tables, dashboard.
Statistics	**Heavily**	Potential outcomes, balance testing, bootstrap intervals, significance of model differences. This is the strongest axis of the project.
Machine learning	Yes	Meta-learners over gradient boosting; uplift forests.
Deep learning	Yes	Shared-representation treatment-effect network (Section 8).
NLP	**No**	No text exists in either dataset. Not simulated.
Computer vision	**No**	No images exist. Not simulated.
Azure AI	Yes	ML endpoint, Blob, ACR, Application Insights.
MLOps / MLflow	Yes	Tracking, registry, champion–challenger, drift monitors, retraining rule.
Hugging Face	Partial	Model/artefact hosting for the public demo; no transformer is warranted by the data and none is used.
Deployment	Yes	FastAPI, Docker, Azure endpoint, CI/CD.
Git & GitHub	Yes	Branching strategy, reviewed pull requests, Actions.
Documentation	Yes	This specification, per-milestone documents, module docs, README, dataset cards.
Testing	Yes	Unit, data, model-behaviour, integration, API, CI.

Two gaps, declared rather than disguised. There is no computer vision and no NLP in this project, because neither dataset contains images or text and manufacturing a use for them would make the system worse and the story weaker. If the evaluation criteria make coverage of those two components mandatory

rather than desirable, that is a reason to reconsider the project choice *now*, in the proposal window — not a reason to attach an unrelated image classifier in Week 18. Raise it with the instructor at the Week 7 review and get an answer on the record.

Part V

Product and Demonstration

40 User Interface

A Streamlit dashboard with four tabs. The design principle: a marketing manager should reach a decision without being taught what a Qini curve is, while a technical examiner should be able to find every underlying number within two clicks.

40.1 Tab 1 — Campaign Planner

Inputs: customer file or dataset selection, per-contact cost, margin per conversion, budget, and a policy mode. Outputs: number of customers to contact, expected incremental conversions with a confidence interval, expected incremental profit, and a downloadable target list. A prominent line states the comparison in words — for example, "this policy is projected to deliver X more conversions than contacting the same number of customers by response score".

40.2 Tab 2 — Policy Comparison

The simulator output (Section 13): uplift policy, response model, blanket contact and random contact, side by side, as a table and as overlaid cumulative-incremental curves with bootstrap bands. A budget slider moves a vertical line across all curves at once.

40.3 Tab 3 — Model Evidence

Qini curve with confidence band; AUUC and Qini coefficient with intervals; the decile table; the uplift calibration plot; SHAP summary; the negative-control result; and the champion–challenger table. This is the tab that answers examiner questions.

40.4 Tab 4 — Monitoring

Feature and score drift charts, data quality status, the registered model version and its training date, and the results of each simulated campaign replay. The delayed effect-drift panel is mocked from replay output and labelled as such — it shows what deployment monitoring would display, without pretending the signal is live.

40.5 Customer detail view

Selecting a row shows the full response object of Section 12: score, interval, segment label, recommendation, verdict, and the top drivers — with abstention verdicts rendered in a visually distinct style from confident ones.

41 End-to-End Demo Scenario

A concrete walkthrough for the defence. All figures below are *illustrative placeholders for the demo script*, not results: the team fills them from its own evaluation output.

1. **Setup.** "This is a base of 100,000 customers. Contacting one costs 5 currency units. A conversion is worth 200. The budget allows 20,000 contacts."
2. **The naive policy.** Run the response model. It selects the 20,000 customers most likely to convert. The simulator reports the incremental conversions actually attributable to contacting them — a number visibly lower than the raw conversion count in that group, because most of those customers would have converted anyway. The gap between those two numbers is the whole argument of the project, on screen, in one moment.
3. **The uplift policy.** Run NetLift at the same budget. It selects a different 20,000. Report incremental conversions with the interval, and the difference from the response model with its own interval.
4. **The sleeping dogs.** Show the suppression list: customers with confidently negative uplift. State plainly that a conventional campaign would have contacted a portion of them.
5. **Abstention.** Select a thin segment. The system returns `insufficient_evidence` and refuses to make a recommendation. Explain that this is designed behaviour.
6. **The negative control.** Show the permuted-treatment run collapsing to zero uplift. This is the moment that establishes that the pipeline is not fooling itself.
7. **Explainability.** Open one customer, show the drivers, and describe the top-uplift segment in plain language.
8. **Operations.** Show the API docs, the MLflow registry, the drift dashboard, and the CI run — five minutes that convert a modelling exercise into a system.

42 What the Demo Must Prove

Three claims, in this order:

1. The system measures an *effect*, not a probability — evidenced by the negative control and by the divergence between the uplift and response rankings.
2. The effect it measures is *real* — evidenced by confidence intervals that exclude zero on the headline result, computed on a locked test split touched once.
3. The effect is *worth money* — evidenced by the simulator's incremental-profit comparison at a fixed budget.

If all three land, the technical depth of the model family barely needs defending. If the first one does not land, nothing else in the presentation matters.

Part VI

Execution

43 Team Distribution

Five members, five roles, defined so that no two people are blocked on the same thing at the same time and every member owns something they can describe end-to-end in an interview.

The critical path is Aliaa (M1) then Ali (M4). Nothing anyone else produces is trustworthy until the data and the split are valid, and no model can be compared with another until the metric harness with confidence intervals exists.

Member	Responsibilities and technologies	Deliverables
Aliaa Gomaa *Data Engineering & Experiment Validity (M1)*	Ingestion and versioning of both datasets, schema contract, duplicate and null policy, the randomization integrity report, the leakage audit and its automated check, the joint stratified split, the preprocessing artefacts, the feature store layout and the ERD. *Depends on: nothing. Can start on day one, and everyone depends on this.*	Reproducible dataset build, the balance report, the split module with its assertions, dataset cards, data dictionary.
Mohammed Sherif *Uplift Modeling Core (M2)*	The baselines (random, blanket, response model), the S/T/X-learners over LightGBM and XGBoost, the class-transformation demonstration including the unbalanced-design trap, model interfaces. *Depends on: Aliaa's split; Ali's metric harness to judge anything.*	Trained baseline and meta-learner models with their configurations, the champion–challenger table entries.
Mahmoud Nadeem *Team Leader; Advanced Models & Experiment Tracking (M3)*	Uplift Random Forest (KL / Euclidean / χ^2), the shared-representation neural estimator in PyTorch, Optuna search under a Qini objective, the full MLflow tracking and registry setup, and reproducibility discipline. *Depends on: Aliaa's split; Mohammed's model interface.*	Advanced model results, the tuning study, the MLflow project with every run logged, the registered champion.
Ali Alaa *Evaluation, Statistics & Decision Layer (M4)*	The metric harness (Qini, AUUC, uplift@k, cumulative gain), bootstrap intervals, the model comparison procedure, the negative control, the behavioural test suite, the decision layer and policy modes, the abstention logic, SHAP, error analysis and the campaign simulator. *Depends on: Aliaa's split. Must deliver the harness before Mohammed and Mahmoud can claim any result.*	Evaluation library with tests, the significance protocol, the decision engine, the simulator, the error-analysis report.
Mazen Mahmoud *MLOps, Backend & Product (M5)*	FastAPI service and contracts, Docker, Azure deployment and Blob storage, PostgreSQL scoring log, the Streamlit dashboard, drift monitors, CI/CD, integration testing and the repository standards. *Depends on: Ali's decision layer for the policy endpoint; Mahmoud's registry for the model.*	Deployed and documented service, dashboard, monitoring, CI pipeline, README and deployment guide.

The critical path is Aliaa (M1) and Ali (M4), in that order — not the modelling. Until the split is valid, every model is untrustworthy; until the metric harness with intervals exists, no model can be compared to another. A schedule that starts modelling in Week 9 while the harness is still being written produces four weeks of results that have to be recomputed. Both of these must be finished and tested before the analysis-and-design milestone ends.

A note on how this reads in an interview. Every member here owns a component with a sentence attached to it that is worth saying out loud: "I built the check that proves our experiment was actually randomized"; "I found and documented a method that silently breaks on unbalanced treatment assignment"; "I built the test that catches leakage by permuting the treatment indicator"; "I built the layer that decides

not to contact customers our model says we would lose by contacting". Distributing work so that each person gets one of those sentences is a design goal of this table, not an accident of it.

The campaign lifecycle is what makes five roles real. If the project were only "fit an uplift model and plot a Qini curve", two people could do it and three would be decorative — and that is the standard failure of tabular ML capstones. The closed loop of Section 14 adds campaign and assignment entities, holdout management, an export path, outcome ingestion, a measurement job and a state machine, on top of an evaluation workstream that is a genuine statistics project in its own right and a serving surface that is a genuine backend project. Those are not tasks invented to fill a team; they are what the difference between a model and a system actually consists of.

44 Integration Plan

```
ALIAA    (M1) --> versioned dataset + valid split + preprocessing artefacts
     |             (contract: feature matrix X, treatment t, outcome y, fold id)
     v
ALI      (M4) --> evaluation harness + decision layer
     |             (contract: metric(y, uplift, t) -> value + CI)
     v
MOHAMMED (M2) --> baselines + meta-learners  ]  both scored by
MAHMOUD  (M3) --> advanced models + MLflow   ]  Ali's harness, in parallel
     |
     v
MAHMOUD  (M3) --> champion registered in MLflow Model Registry
     |             (contract: model URI + version + input schema)
     v
MAZEN    (M5) --> FastAPI service + dashboard + monitoring + CI/CD
     |          + campaign lifecycle: assignment log, holdout, export,
     |            outcome ingestion, realised-effect measurement (S 14)
     v
ALL --> integration -> testing -> deployment -> demo
```

Three interfaces are frozen early and changed only by agreement: the feature-matrix and fold contract between Aliaa and everyone; the metric-function signature between Ali and Mohammed and Mahmoud; and the scoring response schema of Section 12 between Ali, Mahmoud's registry and Mazen's API.

45 Meeting the DEPI Documentation Requirements

The DEPI documentation guidelines ask for artefacts that were written with software systems in mind — ER diagrams, class diagrams, state diagrams, wireframes. Machine-learning teams routinely produce weak versions of these because they treat them as a formality. Each one below has a genuine referent in this system, and the table exists so that no member has to invent one under time pressure at the design milestone.

Required artefact	What it is in NetLift
Use case diagram	Actors: Marketing Analyst, Data Scientist, Scheduler, and — drawn as an *external, deployment-only* actor — a Campaign Execution System. Use cases: define a replay scenario, score a batch, allocate under budget, reveal outcomes and measure, compare policies, retrain, review drift, inspect a customer explanation. The diagram marks which use cases this project implements and which belong to deployment (Section 15).
Functional requirements	Derived from the use cases; each traceable to an API endpoint and a test.
Non-functional requirements	Batch scoring throughput, single-score latency, reproducibility, availability, auditability of every score, and the requirement that no recommendation is issued without a verdict field.

Required artefact	What it is in NetLift
Software architecture	Layered service architecture: data layer, training pipeline, model registry, scoring service, decision layer, presentation. Described in Section 35 and drawn in Section 57.
ER diagram	Real entities: `customer_snapshot` (features as of a date), `replay_scenario`, `treatment_assignment`, `holdout_membership`, `scoring_run`, `score` (one row per customer per run), `model_version`, `outcome_observation`, `drift_report`. Note that `treatment_assignment` and `holdout_membership` are what make the causal claim auditable — they are the database's record of the experiment.
Logical / physical schema	Tables, keys, indexes on `(campaign_id, customer_id)` and `(scoring_run_id)`, and a stated retention policy.
DFD (context and level 1)	Context: dataset store, analyst, NetLift (and, dashed, a campaign execution system that exists only in the deployment view). Level 1: ingest, validate, split, train, evaluate, register, score, decide, reveal, measure, monitor.
Sequence diagrams	Three worth drawing: (1) batch scoring of a campaign list; (2) policy simulation from the dashboard; (3) drift detection triggering a retraining run.
Activity diagram	The replay workflow from scenario definition through scoring and allocation to outcome reveal and measurement, including the abstention branch.
State diagram	Two: the campaign lifecycle (`defined` → `scored` → `allocated` → `revealed` → `measured`) and the model lifecycle (`registered` → `staging` → `production` → `archived`).
Class diagram	The pipeline object model: `DatasetLoader`, `SchemaValidator`, `LeakageAuditor`, `StratifiedSplitter`, `BaseLearner`, `UpliftEstimator` (with `TLearner`, `XLearner`, `UpliftForest`, `NeuralCATE` as subclasses), `UpliftEvaluator`, `PolicyEngine`, `ScoringService`.
Wireframes / mockups	The four dashboard tabs; produced before the UI is coded, not after.
UI/UX guidelines	Colour and typography rules, and specifically the requirement that the two abstention verdicts are visually distinct from a confident recommendation.
Component diagram	Services and their dependencies: training pipeline, registry, scoring API, dashboard, monitoring, database, blob storage.
Deployment diagram	Azure topology: container registry, managed endpoint, blob storage, PostgreSQL, Application Insights, and the CI/CD path from GitHub.
API documentation	Auto-generated OpenAPI from FastAPI, plus a written guide with examples for each endpoint including both abstention responses.
Testing and validation	The test plan of Part IV, with the test-case document listing scenario, input, expected outcome and result.
Deployment strategy	Environments, image versioning, rollback via registry version pinning, and the retraining cadence.

Draw these during the analysis and design milestone, once, properly, and reuse the same diagrams in the final report and the presentation. Teams that redraw them three times in three styles lose a week they did not have.

46 Key Performance Indicators

Each KPI below states what it measures, how it is measured, what counts as success, when it is first reported and who owns it. Two kinds appear, and they are labelled, because confusing them is how project reports end up making claims they cannot support:
Pass/fail — a condition that is either met or not, with no judgement involved.
Measured, no pre-set target — a number that is reported with a confidence interval. No numeric target is set in advance, because there is no prior result on this data to set one from, and inventing a threshold to be beaten would be exactly the kind of unearned claim this project is built to avoid. Where a success condition exists it is *structural* (an interval excluding zero), never a number pulled from nowhere.

KPI	How it is measured	Success condition	First reported	Owner
K1 — Incremental conversions versus the response model *(the primary KPI)*	Simulated replay (Section 14) on the locked test split: both policies select the same number of customers under the same budget; incremental conversions measured against the withheld slice	*Measured.* The 95% bootstrap interval of the *difference* excludes zero in favour of the uplift policy. An interval containing zero is a reportable finding, not a failed project	M4 (W12–13)	Ali
K2 — Qini coefficient and AUUC	Evaluation harness, on the locked test split, with 1000+ bootstrap resamples within treatment arms; library and settings named in the report	*Measured.* Interval excludes zero, i.e. demonstrably better than random targeting	M4 (W12–13)	Ali
K3 — Incremental profit per unit of spend	Replay output converted with the stated per-contact cost and margin	*Measured.* Reported with an interval and with the cost and margin assumptions named on the same line	M4 (W12–13)	Ali
K4 — Negative control	Full pipeline re-run on a randomly permuted treatment indicator	**Pass/fail.** The 95% interval of the measured uplift contains zero	M4, re-run at M6	Ali
K5 — Calibration	Predicted versus measured incremental conversions per decile in the replay	*Measured.* Differences straddle zero across deciles rather than showing a systematic direction	M5	Ali
K6 — Randomization integrity	Standardized mean difference per covariate, plus the ROC–AUC of a classifier trying to predict treatment from covariates alone	**Pass/fail.** Every $\lvert\text{SMD}\rvert < 0.1$ (the conventional threshold, fixed in config before the test is run) and treatment-predictability AUC near 0.5. Any breach documented rather than smoothed over	M3 (W9–10)	Aliaa
K7 — Leakage control	Automated forbidden-column check in CI, plus K4	**Pass/fail.** The check fails the build when a post-treatment column reaches the model matrix, and a unit test asserts that it does	M4	Aliaa
K8 — Reproducibility	Clean checkout, one documented command, compare against the recorded headline number	**Pass/fail.** Reproduces exactly, from config plus commit hash	M5	Mahmoud
K9 — Serving performance	Load test against the deployed endpoint	*Team-set target,* to be revised once first measured: p95 under 300 ms for a single score; a one-million-row batch inside ten minutes on the project's own hardware	M5	Mazen
K10 — Engineering quality	CI on every pull request: lint, type check, unit, data, model-behaviour, integration and API tests	**Pass/fail.** Green on the merge commit of every pull request	M5–M6	Mazen
K11 — Documentation completeness	Checklist of the DEPI-required artefacts (Section 45)	**Pass/fail.** Every required artefact exists and is current	M7	Team leader

K1 is the project. Everything else supports it. If K1's interval excludes zero, the central claim holds and the rest of the report explains how it was earned. If it contains zero, the report says so, and the project becomes an investigation into why — which is why K1 is scheduled at Milestone 4 and not at the end.

47 Milestones

The seven milestones below follow the official DEPI project plan: Team Formation and Planning (Week 5), Instructor Review (Week 7), Requirements Gathering (Weeks 9–10), Analysis and Design (from Week 12), Implementation, Testing, and Release and Final Presentation (Week 21). *Source: DEPI Project Documentation Guidelines and the trainee project instructions, R5 revision.* Each milestone below is written in the same fixed structure — objective, entry criteria, work breakdown by member, deliverables, definition of done, risks — so that any one of them can be lifted out into its own standalone document without rewriting, which is the next document the team needs after this one. Week numbers are *programme* weeks; map them to calendar dates from the official training schedule.

47.1 Milestone 1 — Team Formation and Planning (Week 5)

Objective. Lock the team, the roles, and the project definition, and produce the Project Proposal that goes to the instructor.

Entry criteria. Project direction agreed; this specification read by all five members.

Work breakdown.

- **All members:** read Sections 2.2, 27 and 28. Every member must be able to explain, unprompted, why accuracy is not used. This is the single most likely question in the final defence and it should not fall to one person.
- **Mahmoud (team leader):** the Project Proposal document, the role table, and the GitHub repository request.
- **Aliaa:** download both datasets, confirm the published row counts and column lists against the actual files, and report any discrepancy immediately.
- **Ali:** draft the KPI list — the measurable definitions of project success.
- **Mazen:** repository skeleton, branching strategy, environment file, CI stub.

Deliverables.

- Project Proposal (objective, description, tools and design, technologies, team members and responsibilities, weekly task plan with deliverables).
- Project plan with milestones and a Gantt chart.
- Task assignment and roles document.
- Initial risk register with mitigations (Section 54).
- KPI definitions: Qini coefficient with interval, incremental conversions at fixed budget versus the response model, incremental profit, scoring latency, pipeline reproducibility.
- Repository initialised with structure, README skeleton and CI running.

Definition of done. Proposal submitted to the instructor; every member can state their own role and the project's core question in one sentence each.

Risks. Roles assigned by preference rather than by dependency, leaving the critical path (Aliaa's data-validity role and Ali's evaluation role) with the least experienced members.

47.2 Milestone 2 — Instructor Review (Week 7)

Objective. Obtain approval and, more importantly, obtain answers to the questions that are expensive to get wrong later. Team membership must be final by the end of this week.

Entry criteria. Proposal submitted; datasets downloaded and opened.

Questions to put to the instructor explicitly and record the answers to.

1. Are computer vision and NLP *required* components of the evaluation, or desirable ones? Section 39 states that this project has neither. This is the last week in which that answer can change the project cheaply.
2. Is a public benchmark dataset acceptable as the primary data source, or is locally collected data expected?
3. Is the project graded per team, and is there anything in the grading that depends on other teams? (Relevant because the mega-project option was declined.)
4. What weight do documentation, testing and deployment carry relative to model results?

Work breakdown.

- **Mahmoud (team leader):** present, capture feedback in writing, circulate the same day.
- **Aliaa:** first pass of the randomization integrity check on Hillstrom, so the review has real evidence in it rather than only a plan.
- **Ali:** a first Qini curve on Hillstrom, however rough — a plot in Week 7 changes the tone of the review.
- **Mohammed, Mahmoud and Mazen:** environment set up and running on all machines; the "it works on my laptop only" problem solved now rather than in Week 15.

Deliverables. Instructor feedback document with suggested improvements and the grading criteria breakdown; updated proposal; final confirmed team membership.

Definition of done. Written feedback received; any scope change from it reflected in an updated plan.

Risks. Vague or verbal-only feedback. Ask for the answers to the four questions above specifically, and write down what was said.

47.3 Milestone 3 — Requirements Gathering (Weeks 9–10)

Objective. Convert the project into a specified system: stakeholders, use cases, functional and non-functional requirements. DEPI additionally requires that the task split across members be presented to the instructor by the end of Week 9.

Entry criteria. Approved proposal; both datasets loading reproducibly.

Work breakdown.

- **Mahmoud (team leader):** stakeholder analysis (marketing analyst, data scientist, campaign operations, finance/audit) and the confirmed task-split submission.
- **Ali:** user stories and use cases; the non-functional requirement that no recommendation is returned without a verdict field, written as a requirement rather than as an implementation detail.
- **Aliaa: the randomization integrity report on both datasets, completed.** This is the milestone's most important technical output. If it fails, the project's assumptions change and the team needs to know in Week 10, not Week 18.
- **Mohammed:** the response-model baseline running end-to-end on Hillstrom.
- **Mahmoud:** MLflow tracking server running and logging its first real runs.
- **Mazen:** API contract drafted from the use cases; wireframes for the four tabs.

Deliverables. Stakeholder analysis; user stories and use cases; functional requirements list; non-functional requirements; the randomization integrity report; the confirmed task split.

Definition of done. Every functional requirement maps to a planned endpoint or dashboard element, and every non-functional requirement has a stated way of being measured.

Risks. Requirements written as a formality and never referred to again. Prevent it by requiring each requirement to carry an identifier that later appears in a test name.

47.4 Milestone 4 — Analysis and Design (from Week 12)

Objective. Produce the full design set, and — the part that matters most — run the experiment that tests the project's central hypothesis while there is still time to react to it.

Entry criteria. Requirements approved; valid split implemented and asserted.

Work breakdown.

- **Aliaa:** final split module with its assertions and tests; preprocessing artefacts versioned; ERD and physical schema.
- **Ali: evaluation harness complete**, including bootstrap intervals and the permuted-treatment negative control, with unit tests against a hand-computed fixture. Nothing downstream is trustworthy before this exists.
- **Mohammed:** T-learner and X-learner running on Criteo; **the uplift-versus-response-model comparison executed and reported** (Section 9).
- **Mahmoud:** class, sequence, activity and state diagrams; tuning framework running.
- **Mazen:** component and deployment diagrams; API skeleton with mocked responses so that the dashboard can be built in parallel.

Deliverables. Problem statement and objectives; use case diagram; software architecture; ERD with logical and physical schema; DFD context and level 1; sequence, activity, state and class diagrams; wireframes and UI/UX guidelines; technology stack; deployment and component diagrams; and the **first end-to-end result with a confidence interval**.

Definition of done. The central comparison has been run and its result — positive, negative or inconclusive — is written down with an interval. A negative result at this point is a successful milestone, not a failed one; it redirects the project into investigating why, which is a stronger report than a positive result nobody interrogated.

Risks. Diagram work expanding to fill the milestone while the hypothesis test slips. Sequence the hypothesis test first inside the milestone, not last.

47.5 Milestone 5 — Implementation

Objective. Build the full system to production standards.

Entry criteria. Design set complete; central comparison result known.

Work breakdown.

- **Aliaa:** full pipeline productionised as modules with logging, exception handling and configuration; the automated leakage check enforced in CI.
- **Mohammed:** all meta-learners finalised on Criteo; the class-transformation trap demonstration produced as a figure.
- **Mahmoud:** uplift forest and the neural estimator; the Optuna study; champion registered in MLflow with its evaluation attached.
- **Ali:** decision layer with all four policy modes; abstention logic; SHAP; the campaign simulator; the error-analysis report.
- **Mazen:** FastAPI service against the real model; Docker; Azure deployment; PostgreSQL scoring log; the four dashboard tabs; drift monitors; CI/CD complete.

Deliverables. Structured, commented, type-hinted, modular source code; coding standards applied consistently; branching strategy and meaningful commit history; CI/CD; README with installation, requirements, configuration and execution instructions; API documentation; a deployed link.

Definition of done. A clean checkout plus one documented command reproduces the headline result; the deployed endpoint answers a real request; the dashboard runs against the deployed model.

Risks. Advanced-tier work starting before the MVP is complete. The scope-control order in Section 49 exists for this milestone specifically.

47.6 Milestone 6 — Testing and Quality Assurance

Objective. Establish that the system is correct, and specifically that its central claim is not an artefact.

Entry criteria. System implemented and deployed.

Work breakdown.

- **Ali:** the full behavioural and invariance suite; the negative control re-run and recorded; bootstrap comparison

of champion against every challenger; the locked test split opened *once* for champion confirmation.

- **Aliaa:** data validation tests; reproducibility test from a clean environment.
- **Mohammed and Mahmoud:** model regression tests pinning expected metric ranges.
- **Mazen:** API contract tests including both abstention verdicts and malformed input; load test of the batch endpoint; integration test end to end.
- **All:** a written test-case document — scenario, input, expected outcome, actual outcome — and a bug log with resolutions.

Deliverables. Test plan and test cases; automated test scripts; bug reports and resolutions; the final metric table with confidence intervals; the champion–challenger comparison.

Definition of done. CI green; negative control passes; the locked test result is recorded and is not revisited afterwards.

Risks. Reopening the test split after a disappointing number and re-tuning against it. Write down, before opening it, that it is opened once — and hold to it. This is the discipline that makes the final number mean anything.

47.7 Milestone 7 — Release and Final Presentation (Week 21)

Objective. Deliver, document, and defend.

Entry criteria. Testing complete; results final.

Work breakdown.

- **All:** rehearse the demo of Part V at least twice end to end, on the deployed system, with the network involved.
- **Mahmoud (team leader):** final upload of all project files to the assigned repository; the presentation; the per-member contribution slide the final evaluation explicitly requires.
- **Ali:** the results section of the final report, with intervals on every number.
- **Mazen:** user manual; deployment guide; recorded demo video as a fallback in case live infrastructure fails on the day.
- **Aliaa, Mohammed, Mahmoud:** technical documentation for their components.

Deliverables. User manual; technical documentation covering architecture, schema and API; project presentation; demo (live, with a recorded backup); final report; complete repository.

Definition of done. Every member can answer "what did you build?" with a specific component and a specific technical decision they made inside it. The final DEPI evaluation is explicitly based on the team demonstrating the work completed and each individual's responsibility in it, so this is a graded requirement, not a courtesy.

Risks. A live demo failing on the day. Mitigate with the recorded video and a local fallback mode that runs without the cloud endpoint.

48 Project Timeline

Programme weeks, aligned to the DEPI milestone plan.

Weeks	Milestone	Work
5	M1 Planning	Team and roles locked; proposal; repository and CI skeleton; datasets downloaded and their published figures verified against the files.
6	—	Environment working on every machine; Hillstrom loading; first exploratory analysis.
7	M2 Instructor review	Present; get the four questions answered in writing; team membership final; first rough Qini curve on Hillstrom.
8	—	Split module drafted; response-model baseline on Hillstrom.
9–10	M3 Requirements	Requirements set; **randomization integrity report on both datasets**; task split submitted; API contract and wireframes.
11	—	Criteo loading at full scale; memory and format decisions settled.
12–13	M4 Analysis & design	Full design and diagram set; **evaluation harness with bootstrap intervals and negative control**; **uplift versus response model, first result with an interval**.
14–15	M5 Implementation	Meta-learners finalised on Criteo; decision layer and policy modes; FastAPI service against the real model.
16–17	M5 Implementation	Advanced tier: uplift forest, neural estimator, Optuna study; simulator; dashboard tabs 1–3; Azure deployment.
18	M5 Implementation	Multi-treatment and revenue uplift on Hillstrom if on schedule; drift monitors; dashboard tab 4; CI/CD complete.
19–20	M6 Testing	Full test suite; behavioural and invariance tests; champion confirmed once on the locked split; error analysis; bug log.
21	M7 Release	Final report, user manual, technical documentation, presentation, rehearsed demo, repository upload.

The two dates to protect above all others. The randomization integrity report by the end of Week 10, and the uplift-versus-response-model comparison by the end of Week 13. Everything else in this timeline can slip by a week without damage. If either of those two slips, the project loses the ability to react to what it finds, and a project that cannot react to its own findings is reduced to reporting whatever it happens to get.

49 Scope Control

49.1 MUST HAVE

- Randomization integrity report on both datasets
- Automated leakage audit, enforced by a test
- Joint stratified split on (treatment, outcome), asserted
- All three baselines: random, blanket, response model
- T-learner and X-learner over gradient-boosted trees
- Qini, AUUC and uplift@k with bootstrap confidence intervals
- The permuted-treatment negative control
- Decision layer with negative-uplift suppression and both abstention verdicts
- Campaign policy simulator with incremental-profit output
- MLflow tracking and registry; reproducible from config plus commit
- FastAPI service, Dockerised, deployed; Streamlit dashboard
- Test suite covering unit, data, model behaviour, integration and API

49.2 SHOULD HAVE

- Uplift Random Forest as a model-class challenger
- Optuna tuning under a Qini objective, with trial counts reported
- SHAP explainability and the persuadable-segment profile
- Azure ML managed endpoint (rather than a container alone)
- Drift monitors and a documented retraining rule

49.3 NICE TO HAVE

- Shared-representation neural estimator
- Multi-treatment uplift on Hillstrom's two arms
- Revenue (continuous-outcome) uplift on `spend`
- Causal forest with interval estimates
- The biased-versus-corrected class-transformation figure

49.4 REMOVE IF BEHIND SCHEDULE, in this order

1. Causal forest
2. Multi-treatment and revenue uplift
3. The neural estimator — cut without hesitation; it is a challenger, not the project
4. Uplift Random Forest
5. Azure managed endpoint, falling back to a container with a documented deployment path

Never cut, at any level of schedule pressure: the confidence intervals, the negative control, the locked test split discipline, and the response-model baseline. Those four are what separate this from a project that reports a number nobody can trust. Everything else on the lists above is negotiable; these four are not.

50 Business Model

Model	Assessment
B2B SaaS for mid-size retail / e-commerce	**The most realistic near-term fit.** These companies run campaigns constantly and have no in-house causal-inference capability; the value is immediate and measurable in their own reporting.
Enterprise licence (banks, telecoms)	Highest value per contract, longest sales cycle. These organisations already have data science teams, so the sale is a capability accelerator rather than a replacement. Realistic only after a reference deployment exists.
API / usage-based	Natural given the architecture, and the lowest-friction way to start; pricing per scored customer or per campaign.
Consulting engagement	The most honest first revenue for a student team: run one campaign for one company, measure it against a real holdout, and let the result be the case study.
Embedded in a CRM or marketing-automation platform	The largest eventual market and the least accessible one; a partnership route, not a starting point.

The sales sentence, and it is unusually strong for a student project: "Give us a randomized holdout on one campaign. If we do not produce more incremental conversions per unit of spend than your current targeting, you owe nothing." The product is measurable by construction — the same property that makes it a good capstone makes it a good commercial pitch.

51 Competitive Advantage

Compared with	Difference
Response / propensity models	Answers a different question. Cannot separate persuadables from customers who would have converted anyway; NetLift's entire purpose is that separation.
A/B testing alone	Produces one average verdict for the whole population; NetLift produces a per-customer policy.
Rule-based segmentation	Encodes a marketer's prior beliefs; NetLift estimates the effect from data and can contradict those beliefs, including in the direction of suppression.
Off-the-shelf marketing-automation scoring	Optimises predicted engagement, which is the same mistake in a different interface, and is opaque about whether contact caused anything.
Generic churn dashboards	Report who is at risk, not what to do about it, and never report whether the intervention worked.

52 Limitations

Stated explicitly, because a limitations section that is thin is read as a limitations section that was not thought about.

1. **Public benchmark data, not Egyptian market data.** The results describe the populations in the Criteo and Hillstrom experiments. The *method* transfers to any market; the *numbers* do not. Say this before being asked.
2. **Anonymised features on the primary dataset.** Criteo's features carry no domain meaning, so segment descriptions on that dataset can only be structural. Interpretable segment profiling is possible on Hillstrom alone.
3. **One intervention, one context.** The estimated effects belong to the specific offer, channel and time period of each experiment; they do not generalise to other offers.
4. **Small effects, wide intervals.** With a conversion rate near 0.29%, some segment-level estimates will be inconclusive. That is a property of the problem, and the abstention mechanism is the honest response to it.
5. **Short outcome windows.** Hillstrom measures a two-week window. An offer can move a purchase forward in time without creating an additional purchase — an effect that a two-week window reads as positive uplift and a one-year window might not.
6. **SUTVA is assumed, not tested.** Customer-to-customer spillover would bias the estimates and cannot be detected in this data.
7. **The replay is not a campaign.** Replaying decisions over recorded outcomes removes every operational hazard a real campaign has: no delivery failures, no channel contamination, no population shift between scoring and contact.
8. **Intention-to-treat is the headline estimand, and the treatment-on-the-treated estimate describes a small, self-selected group.** The reported estimates describe the effect of being *selected* for the campaign, not of having received the intervention. Hillstrom carries no exposure indicator at all. Criteo does, so a treatment-on-the-treated estimate is available there as a Wald ratio: 3.20 percentage points among compliers, a 146% relative increase over their own untreated baseline of 2.18% (Section 6). Two things limit it. It rests on the exclusion restriction, which cannot be tested here. And the compliers are the 3.60% of treated users who were actually shown the advertisement, whose untreated baseline is about eighteen times the never-takers' — so the estimate describes a strongly self-selected minority and says nothing about the other 96%. An earlier version of this document called the same estimate "not credible"; that judgement came from dividing it by the wrong baseline and has been withdrawn.
9. **Never validated against a live campaign.** The system is measured by replaying decisions over a held-out slice of historical experimental data (Section 14). It has never scored a live audience, and the mechanisms that would make live measurement possible — a permanent randomized holdout, outcome-window measurement — are designed but not exercised (Section 15).
10. **No computer vision or NLP** (Section 39).

53 Ethical and Privacy Considerations

- **Both datasets are public and de-identified.** No personal data is collected, processed or published by this project, and the repository contains loader scripts and checksums rather than raw data.
- **Uplift modelling is behaviour modification with a measured effect size** — that is literally what it estimates. Applied to a discount it is ordinary marketing. Applied to manipulative retention tactics, to targeting customers identified as financially vulnerable, or to persuading someone toward a product that harms them, the same machinery becomes something else. The report should state a scope of acceptable use, not treat the question as outside its remit.
- **Differential treatment by segment** means some customers systematically receive offers that others do not. Where the features encode or proxy protected characteristics, the policy can become discriminatory even though no such feature was consciously used. **Recommended deliverable:** report targeting rates across available demographic-adjacent segments, and state that the check was performed — on Criteo the features are anonymised and this can only be done structurally, which should also be stated.
- **The holdout has an ethical dimension too.** Withholding an offer from a control group is standard practice and is what makes measurement possible; it should still be named and justified rather than passed over silently.
- **Transparency of the abstention path.** A system that says "we do not know" is more honest than one that always answers, and the interface must not hide that state to look more decisive.

54 Risks and Difficulties

Risk	Severity	Mitigation
The central hypothesis does not hold — uplift targeting does not beat the response model on this data	High	Run the comparison by Week 13, not Week 19. If it holds, it is the headline. If it does not, the project becomes an investigation of *why*, which is a legitimate and frequently stronger result. Neither outcome is a failed project; only discovering it too late is.
Effects too small to separate from noise	High	Develop against `visit` (rate 0.046992) as well as `conversion` (0.00292); report both; use the abstention path rather than reporting an inconclusive number as a finding.
Curriculum gap: no CV, no NLP	Medium–High	Raise it explicitly at the Week 7 review and get the answer in writing. This risk is cheap to resolve in Week 7 and expensive in Week 18.
Uneven team workload — a tabular ML project can leave two members working and three watching	Medium–High	This is a real and well-known failure mode for exactly this class of project, and it is why Section 43 places statistics and evaluation, the decision layer and simulator, and the full MLOps and product surface as three independent workstreams rather than as appendices to the modelling. Enforce it by requiring each member to own a component that appears in the demo.
Metric overfitting — tuning against the test split	Medium	Locked test split, opened once, with the rule written down before it is opened.
Silent leakage	Medium	Automated forbidden-column check; the permuted-treatment negative control, which catches what the check misses.
Cross-fitting implemented incorrectly in the X- or R-learner	Medium	Use the library implementation; write a test asserting fold membership; do not hand-roll.
Scale problems on 14M rows	Low–Medium	Parquet with column subsets, chunked preprocessing, stratified subsample for iteration, full data only for reporting.

Risk	Severity	Mitigation
Azure credit exhausted	Low	Train locally; shut down compute after every session; budget alert set well below the limit.
Live demo failure	Low	Recorded demo video and a local fallback mode.

54.1 Top three

1. The uplift-versus-response-model comparison returning a null result, discovered too late to investigate.
2. Confidence intervals on the headline result straddling zero, leaving the project with no statement it can defend.
3. The curriculum gap on computer vision and NLP turning into an evaluation penalty that was foreseeable in Week 7.

All three are mitigated by the same discipline: *find out early.* Every one of them is cheap in Week 10 and expensive in Week 19.

55 Anticipated Defence Questions

The questions a competent examiner will ask, and where each is answered. Every member should be able to handle the first four regardless of which component they own.

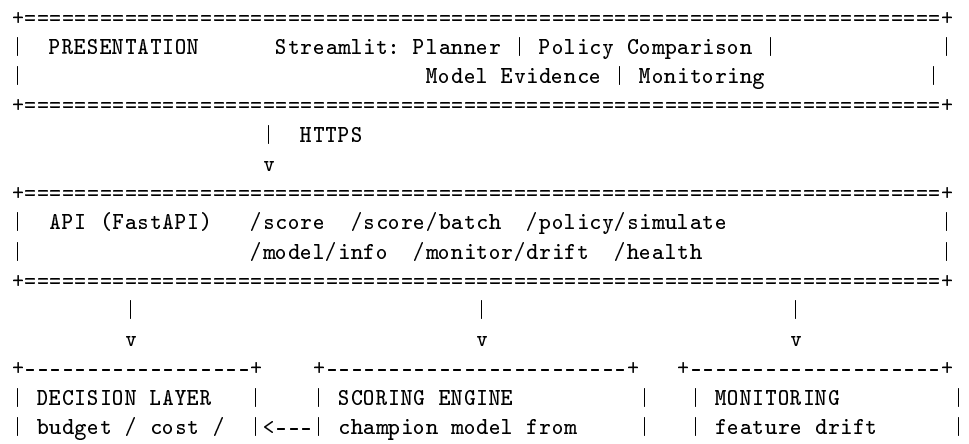
Question	Answer, and where it lives
"What was your accuracy?"	There is none, and the reason is structural: the target is a difference between two potential outcomes, only one of which is ever observed (Section 2.2). We report Qini, AUUC and uplift@k with confidence intervals (Section 27).
"How do you know the effect you measured is not noise?"	Bootstrap intervals within treatment arms on every reported figure, a bootstrap comparison of champion against challenger, and a permuted-treatment negative control that collapses to zero (Section 28).
"How is this different from a churn model?"	A churn model ranks by predicted outcome and cannot separate a customer who converts because of the contact from one who would have converted anyway. The response-model baseline is in the project precisely so that difference is measured rather than asserted (Section 9).
"Is your campaign loop real?"	No, and the specification says so on the first page of Section 14. It is a simulated replay over a held-out split: the system decides without reading outcomes, the outcomes are then revealed, and the decision is scored against them. A live loop would need the holdout of Section 15.
"Why is a public dataset acceptable?"	Because the assumption the entire method rests on — that treatment was randomly assigned — is guaranteed by these datasets' experimental design and verified by our own balance report. Observational data would require assumptions we could not check (Section 5).
"Where is the deep learning?"	A shared-representation treatment-effect network, included as a challenger with a problem-based justification, and honestly reported if the gradient-boosted baseline beats it (Section 8).
"Where is the computer vision / NLP?"	Absent, deliberately, because neither dataset contains images or text. Stated in the specification rather than disguised (Section 39).

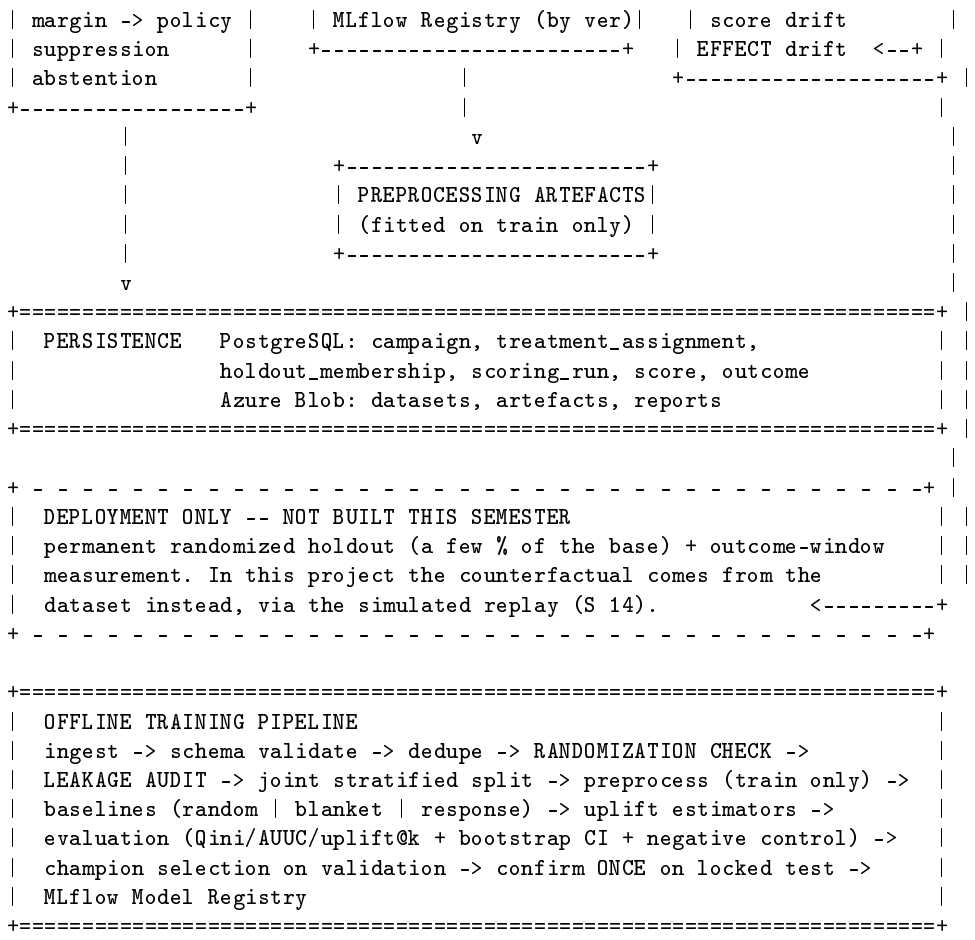
Question	Answer, and where it lives
"What happens after deployment — how do you know it still works?"	In deployment: a permanently maintained randomized holdout plus drift monitors and a documented retraining rule (Sections 32, 15). We designed it; we did not exercise it, because historical data gives us the counterfactual directly.
"Did you tune on your test set?"	No. Selection on validation, one confirmation pass on a locked test split, and the rule written down before the split was opened (Section 18).
"What if the model is wrong about a customer?"	The system returns an explicit `insufficient_evidence` or `insufficient_control_data` verdict instead of a recommendation whenever the interval straddles zero or the control support is too thin (Section 11).
"What did *you* personally build?"	Answered per member from Section 43; each role owns a component that appears in the demo.

56 Final Deliverables

- Reproducible data pipeline with schema validation, leakage audit and the split module
- Randomization integrity report for both datasets
- Trained models: three baselines, T- and X-learners, and the Advanced-tier challengers
- Evaluation library: Qini, AUUC, uplift@k, cumulative gain, bootstrap intervals, negative control, behavioural tests
- Decision engine with four policy modes and the abstention logic
- Campaign policy simulator
- MLflow project with every run logged and the champion registered
- FastAPI service, Dockerised and deployed, with OpenAPI documentation
- Streamlit dashboard, four tabs
- Drift monitoring and a documented retraining rule
- Full test suite and CI pipeline
- Documentation set: this specification, the seven per-milestone documents, requirements and design documents with the full diagram set, dataset cards, data dictionary, README, user manual, technical documentation, test plan and test cases, bug log
- Final report, presentation, and demo with a recorded backup
- Complete repository on the assigned organisation account

57 Final Architecture Diagram





58 Final Evaluation

58.1 Scores

Dimension	Score	Justification
Technical depth	8/10	Causal estimation, cross-fitting, bootstrap inference and a production service; deep without being exotic.
AI/ML depth	9/10	The estimand is not observable, which changes the estimator design, the metric set and the validation strategy at once. Very few student projects operate under that constraint.
Statistical rigour	10/10	The strongest axis by a distance: potential outcomes, balance testing, bootstrap intervals, negative controls, locked-split discipline.
Deep learning	6/10	Present and problem-motivated, but a challenger rather than the core, and honestly scored as such.
Data engineering	7/10	Strong pipeline discipline at 14M rows, but the data is pre-collected; there is no collection operation as in earlier projects considered.
MLOps / production	9/10	Registry, versioned scoring, drift monitoring, retraining rule, CI/CD, and a permanent production holdout — a genuinely complete lifecycle story.
Business value	9/10	Directly monetary, directly measurable, in the language a finance function uses.

Dimension	Score	Justification
Employer signal	9/10	Retention and campaign targeting are live problems in exactly the banking, telecom and FinTech employers this track feeds.
Innovation	6/10	Uplift modelling is an established field and this project does not extend it. Its originality lies in rigour and in the completeness of the system, not in method — and claiming otherwise would be the wrong move in a defence.
Portfolio value	8/10	The interview answer — "the label we were predicting does not exist, so here is how we validated anything at all" — is unusually strong.
Demo quality	6/10	A dashboard with charts and currency figures. Legible and persuasive to a business audience, but it does not move on screen; weaker on immediate impact than a live camera demo, and honestly scored.
Feasibility	9/10	Both datasets are public, verified and downloadable today; no external dependency, no permissions, no hardware constraint.
Overall	**8.0/10**	Mean of the above.
Computer vision	*n/a*	Not present. Excluded from the mean rather than scored, and flagged as a curriculum risk in Section 54 rather than concealed.
NLP	*n/a*	As above.
Difficulty *(not a quality score)*	7/10	Conceptually demanding, engineering-wise moderate. The difficulty is in reasoning correctly, not in making things run.

58.2 Top strengths

The problem is genuinely different from the classification projects that dominate this cohort, and the difference is easy to state in one sentence. The business value is monetary and measurable. The data is verified, public and available today, so feasibility risk is close to zero. The statistical discipline the problem forces — intervals, negative controls, locked splits — is exactly the discipline that distinguishes an engineer from a notebook operator, and the supervising engineer's criterion of commercial relevance is met directly rather than argued for.

58.3 Top weaknesses

No computer vision and no NLP, which is a real gap against the track's breadth. Innovation is moderate: this is a well-established field and the project's contribution is rigour, not method. Demo quality is good but static — there is no moment of visual surprise. And the results describe public benchmark populations, not the Egyptian market, which limits how far the local-relevance argument can be pushed.

58.4 Do I recommend building it?

Yes — with one condition attached. The project is well-scoped, the data risk is essentially zero, the commercial framing matches the guidance the team was given, and the technical core is a real problem with real depth rather than a classification task in costume. The condition is Section 54: get an answer on the computer-vision and NLP question at the Week 7 review. That is the only issue in this document that could still change the project's viability, and it is answerable in one conversation.

58.5 Is the scope realistic for five students?

Yes, for the MVP defined in Section 49, and only if the role split in Section 43 is followed. The failure mode for a project of this shape is not that it is too hard — it is that two members do the modelling and three find nothing substantial to own. The evaluation and statistics workstream, the decision layer and simulator, and the MLOps

and product surface are what make five roles real rather than nominal.

58.6 What to remove if the project becomes too large

In the order given in Section 49: causal forest, then multi-treatment and revenue uplift, then the neural estimator, then the uplift forest, then the managed cloud endpoint. Never the intervals, the negative control, the locked split, or the response-model baseline.

59 Final Recommended Version

1. **Project name:** NetLift.
2. **Problem statement:** Retention and acquisition budgets are allocated by models that predict who is likely to convert, which cannot distinguish customers who convert *because* of the contact from customers who would have converted anyway — and cannot detect the customers who convert less *because* they were contacted.
3. **MVP:** single-treatment, binary-outcome uplift estimation on randomized experimental data, with three mandatory baselines, a Qini/AUUC evaluation harness carrying bootstrap intervals and a permuted-treatment negative control, a budget-constrained decision layer with negative-uplift suppression and two abstention verdicts, a campaign policy simulator reporting incremental profit, and a deployed scoring service with a dashboard.
4. **Architecture:** validated experimental data → randomization and leakage gates → joint stratified split → baselines → meta-learner uplift estimators → evaluation with intervals → decision layer → FastAPI service and Streamlit dashboard → drift monitoring against a permanent randomized holdout.
5. **What makes it a system rather than a model:** the simulated campaign replay of Section 14 — define a scenario, score, allocate under budget, hold out, reveal the recorded outcomes, measure what the decision produced, compare against the baseline policies. It measures the decision rather than the ranking, and it is what makes five roles necessary rather than nominal. What a live deployment would additionally need is stated separately in Section 15 and is not claimed as project work.
6. **Data:** Criteo-UPLIFT as the primary result (13,979,592 rows, 12 features, treatment ratio 0.85); Hillstrom MineThatData for development, multi-treatment and revenue uplift (64,000 customers, three equal arms).
7. **Models:** response-model baseline, T-learner and X-learner over gradient boosting as the MVP, with the X-learner favoured because of the 85/15 treatment imbalance; uplift forest and a shared-representation neural estimator as challengers.
8. **Technology:** Python, pandas, LightGBM/XGBoost, scikit-uplift, CausalML, PyTorch, SHAP, Optuna, MLflow, FastAPI, Docker, PostgreSQL, Azure ML, Streamlit, GitHub Actions.
9. **Team:** Aliaa Gomaa (Data Engineering and Experiment Validity) / Mohammed Sherif (Uplift Modeling Core) / Mahmoud Nadeem (Team Leader; Advanced Models and Tracking) / Ali Alaa (Evaluation, Statistics and Decision Layer) / Mazen Mahmoud (MLOps, Backend and Product).
10. **Timeline:** DEPI programme weeks 5 to 21, with the randomization report due by Week 10 and the central hypothesis test by Week 13.
11. **Main risks:** a null result on the central comparison; intervals that straddle zero; the computer-vision and NLP curriculum gap; and uneven workload if the lifecycle work is dropped.
12. **Explicitly not in scope this semester:** observational (non-randomized) causal inference; continuous-dose treatment; online policy learning; cross-campaign budget allocation.
13. **Why this balance is the right one:** it is the only direction considered in this selection process where the hardest technical idea in the project (an unobservable target) is also the thing that produces the clearest commercial argument (money saved per campaign) and the cleanest defence answer ("here is how we proved it was not noise"). The cost of that alignment is breadth — no vision, no language — and the document has stated that cost plainly rather than disguising it.

This specification was prepared as a technical architecture review. Dataset statistics are taken from the sources cited at the point of use and should be re-verified against the downloaded files before being quoted in the team's own submitted documentation; figures marked as derived are arithmetic consequences of those published values, not independent measurements. Design decisions, scores and risk assessments are analytical judgements offered to inform the team's decisions, not substitutes for them. Sections marked with a red border are the ones most likely to be probed in a defence.